analysis

March 20, 2026

1 EEG Analysis: AI-generated vs Human Stimulus (Neutral + Happy)

This notebook builds a full pipeline for 8-channel EEG recorded with the following 10–20 mapping:

- 1 → Fz
- 2 → C3
- 3 → Cz
- 4 → C4
- 5 → T5
- 6 → Pz
- 7 → T6
- 8 → O1 (recorded as posterior occipital channel)

Reference note: - Recording reference was external (`O2/O2` in experiment notes), so it is **not present as a measured 9th signal** in `eeg.npy`. - Therefore we cannot directly use it as a data channel; instead we re-reference all recorded channels to **common average reference (CAR)**.

Goals: 1. Compare AI vs Human responses (using only Neutral and Happy scenarios). 2. Check whether frontal theta (Fz, 4–8 Hz) increases for AI stimuli. 3. Identify which channels/regions contribute most to AI-vs-Human decoding. 4. Handle bad channels robustly at trial level.

[ ]:
```
import os
import json
from collections import Counter, defaultdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import mne
from mne.preprocessing import ICA

from scipy.signal import butter, iirnotch, sosfiltfilt, filtfilt, welch,␣
↪spectrogram, detrend, resample
from scipy.stats import ttest_ind, kurtosis
```

```python
from sklearn.model_selection import StratifiedKFold, GroupKFold,␣
↪cross_validate, cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, f1_score

sns.set_theme(style='whitegrid')
np.set_printoptions(precision=3, suppress=True)

DATA_ROOT = './data'

# Restrict analysis to this subset only
TARGET_SUBJECT = 'sub-03'
TARGET_SESSION = 'ses-01'

# 8-channel mapping provided
CH_NAMES = ['Fz', 'C3', 'Cz', 'C4', 'T5', 'Pz', 'T6', 'O1']

# Keep only these conditions
KEEP_CATEGORIES = {'AI', 'Human'}
KEEP_SCENARIOS = {'Neutral', 'Happy'}

# Spectral bands used for features
BANDS = {
    'delta': (1, 4),
    'theta': (4, 8),
    'alpha': (8, 13),
    'beta': (13, 30),
    'gamma': (30, 40),
}

# Acquisition sampling rate is fixed at 250 Hz
TARGET_FS = 250.0
TARGET_EPOCH_SEC = 5.0   # fixed epoch length from stimulus onset
TARGET_N_SAMPLES = int(round(TARGET_FS * TARGET_EPOCH_SEC))
BASELINE_SEC = 0.2       # baseline window at trial start (proxy if no pre-stim␣
↪in saved EEG)

# Independent final holdout split (never used for CV/model selection)
FINAL_TEST_SIZE = 0.20

# ICA settings (extended Infomax, as used in test2)
USE_EXTENDED_INFOMAX_ICA = True
ICA_MAX_COMPONENTS = 7
ICA_KURTOSIS_Z = 2.0
```

```
ICA_MAX_EXCLUDE = 2

# Manual ICA selection controls
ICA_SHOW_PLOTS = True
ICA_PLOT_SECONDS = 20
ICA_USE_MANUAL_SELECTION = True
# Example key format: 'sub-03/ses-01/run-01': [0, 3]
ICA_MANUAL_EXCLUDE = {
    # 'sub-03/ses-01/run-01': [0],
    # 'sub-03/ses-01/run-02': [0],
}

# Detailed decomposition views
ICA_SHOW_DECOMPOSITION = True
ICA_MAX_DECOMP_COMPONENTS = 7

RANDOM_STATE = 42
```

[2]:
```
def estimate_fs(ts):
    if len(ts) < 2:
        return np.nan
    return (len(ts) - 1) / (ts[-1] - ts[0])


def build_run_index(data_root=DATA_ROOT):
    rows = []
    for sub_dir in sorted(os.listdir(data_root)):
        if not sub_dir.startswith('sub-'):
            continue
        sub_path = os.path.join(data_root, sub_dir)
        if not os.path.isdir(sub_path):
            continue
        for ses_dir in sorted(os.listdir(sub_path)):
            if not ses_dir.startswith('ses-'):
                continue
            ses_path = os.path.join(sub_path, ses_dir)
            for run_dir in sorted(os.listdir(ses_path)):
                if not run_dir.startswith('run-'):
                    continue
                run_path = os.path.join(ses_path, run_dir)
                eeg_file = os.path.join(run_path, 'eeg.npy')
                ts_file = os.path.join(run_path, 'timestamp.npy')
                markers_file = os.path.join(run_path, 'markers.json')
                meta_file = os.path.join(run_path, 'metadata.json')

                if not (os.path.exists(eeg_file) and os.path.exists(ts_file)␣
 ↪and os.path.exists(markers_file)):
```

```
                    continue

                eeg = np.load(eeg_file)
                ts = np.load(ts_file)
                with open(markers_file, 'r') as f:
                    markers = json.load(f)
                meta = {}
                if os.path.exists(meta_file):
                    with open(meta_file, 'r') as f:
                        meta = json.load(f)

                rows.append({
                    'subject': sub_dir,
                    'session': ses_dir,
                    'run': run_dir,
                    'run_path': run_path,
                    'eeg_shape': tuple(eeg.shape),
                    'n_markers': len(markers),
                    'fs_est': estimate_fs(ts),
                    'metadata': meta,
                })

    return pd.DataFrame(rows)


run_df = build_run_index(DATA_ROOT)
run_df = run_df[(run_df['subject'] == TARGET_SUBJECT) & (run_df['session'] ==␣
 ↪TARGET_SESSION)].copy()

print(f'Runs discovered in subset ({TARGET_SUBJECT}/{TARGET_SESSION}):␣
 ↪{len(run_df)}')
if len(run_df):
    display(run_df[['subject', 'session', 'run', 'eeg_shape', 'n_markers',␣
 ↪'fs_est']])
    print('\nEstimated fs summary:')
    print(run_df['fs_est'].describe())
else:
    print('No runs found for requested subset.')
```

```
Runs discovered in subset (sub-03/ses-01): 2
```

	subject	session	run	eeg_shape	n_markers	fs_est
4	sub-03	ses-01	run-01	(8, 89701)	60	250.242336
5	sub-03	ses-01	run-02	(8, 89581)	60	250.607543

```
Estimated fs summary:
count      2.000000
```

```
mean     250.424939
std        0.258241
min      250.242336
25%      250.333637
50%      250.424939
75%      250.516241
max      250.607543
Name: fs_est, dtype: float64
```

[17]:
```
def ensure_ch_by_time(eeg):
    """Return EEG as (n_channels, n_samples)."""
    eeg = np.asarray(eeg)
    if eeg.ndim != 2:
        raise ValueError(f'Expected 2D EEG array, got shape={eeg.shape}')

    # In this project we expect 8 channels. If first axis is 8, keep it.
    if eeg.shape[0] == len(CH_NAMES):
        return eeg
    if eeg.shape[1] == len(CH_NAMES):
        return eeg.T

    # Fallback: assume smaller dimension is channels.
    return eeg if eeg.shape[0] < eeg.shape[1] else eeg.T


def run_extended_infomax_ica(eeg, fs, ch_names, run_key='run'):
    """
    Run extended Infomax ICA on continuous run data.
    Visualize IC topographies and source signals so user can manually choose␣
↪artifacts.
    Returns cleaned EEG and excluded IC indices.
    """
    if not USE_EXTENDED_INFOMAX_ICA:
        return eeg, []

    n_ch = eeg.shape[0]
    n_comp = min(ICA_MAX_COMPONENTS, n_ch - 1)
    if n_comp < 2:
        return eeg, []

    # Map legacy 10-20 names to modern montage aliases for plotting
    ch_name_map = {'T5': 'P7', 'T6': 'P8'}
    ch_names_plot = [ch_name_map.get(c, c) for c in ch_names]

    info = mne.create_info(ch_names=ch_names_plot, sfreq=fs, ch_types=['eeg'] *␣
↪n_ch)
    raw = mne.io.RawArray(eeg, info, verbose=False)
```

```
montage = mne.channels.make_standard_montage('standard_1020')
raw.set_montage(montage, on_missing='ignore')

# Keep ICA preprocessing aligned with rest of pipeline
raw.filter(l_freq=1.0, h_freq=40.0, fir_design='firwin', verbose=False)
raw.notch_filter(freqs=60.0, verbose=False)
raw.set_eeg_reference('average', projection=False, verbose=False)

ica = ICA(
    n_components=n_comp,
    method='infomax',
    fit_params=dict(extended=True),
    random_state=RANDOM_STATE,
    max_iter=2000,
)
ica.fit(raw, verbose=False)

# Suggested ICs via robust kurtosis (for convenience)
src = ica.get_sources(raw).get_data()  # (n_components, n_samples)
mixing = ica.mixing_matrix_            # (n_channels, n_components)

k = np.abs(kurtosis(src, axis=1, fisher=True, bias=False))
kz = (k - np.median(k)) / (np.std(k) + 1e-12)
suggested = np.where(kz > ICA_KURTOSIS_Z)[0].tolist()[:ICA_MAX_EXCLUDE]

if ICA_SHOW_PLOTS:
    print(f"\n=== ICA review: {run_key} ===")
    print('Suggested artifact ICs (kurtosis heuristic):', suggested)

    # 1) Topographic component maps (like test2)
    try:
        ica.plot_components(picks=range(n_comp), inst=raw)
        plt.suptitle(f'ICA components (extended Infomax) | {run_key}', y=1.
↪02)
        plt.show()
    except Exception as e:
        print('Topomap visualization failed; showing mixing matrix instead:
↪', e)
        mm = mixing
        plt.figure(figsize=(8, 4))
        plt.imshow(mm, aspect='auto', cmap='RdBu_r')
        plt.colorbar(label='Mixing weight')
        plt.yticks(np.arange(len(ch_names_plot)), ch_names_plot)
        plt.xticks(np.arange(mm.shape[1]), [f'IC{i}' for i in range(mm.
↪shape[1])])
        plt.title(f'ICA mixing matrix | {run_key}')
```

```
            plt.xlabel('Components')
            plt.ylabel('Channels')
            plt.tight_layout()
            plt.show()

        # 2) Source time courses browser (like test2)
        ica.plot_sources(raw, show_scrollbars=False, title=f'ICA source signals␣
↪| {run_key}')
        plt.show()

        # 3) Static stacked source traces for quick manual inspection
        n_plot = min(int(ICA_PLOT_SECONDS * fs), src.shape[1])
        t = np.arange(n_plot) / fs
        plt.figure(figsize=(12, 6))
        for ic in range(src.shape[0]):
            plt.plot(t, src[ic, :n_plot] + ic * 15, lw=1)
        plt.xlabel('Time (s)')
        plt.ylabel('IC signal + offset')
        plt.title(f'ICA source traces (first {ICA_PLOT_SECONDS}s) | {run_key}')
        plt.grid(alpha=0.3)
        plt.show()

        # 4) Decomposition diagnostics: component->channel contributions
        if ICA_SHOW_DECOMPOSITION:
            n_decomp = min(ICA_MAX_DECOMP_COMPONENTS, n_comp)

            # Heatmap: channels x components mixing weights
            plt.figure(figsize=(8, 4))
            sns.heatmap(
                mixing[:, :n_decomp],
                cmap='RdBu_r',
                center=0,
                yticklabels=ch_names,
                xticklabels=[f'IC{i}' for i in range(n_decomp)],
            )
            plt.title(f'ICA mixing weights (channel contributions) | {run_key}')
            plt.xlabel('ICA Component')
            plt.ylabel('Channel')
            plt.tight_layout()
            plt.show()

            # For each component: source + channel weights + reconstructed␣
↪per-channel traces
            for ic in range(n_decomp):
                comp = src[ic, :n_plot]                    # s_k(t)
                weights = mixing[:, ic]                    # a_{c,k}
```

```
                recon = weights[:, None] * comp[None, :]   # contribution per␣
↪channel

                fig, axes = plt.subplots(1, 3, figsize=(18, 4))

                # Source signal
                axes[0].plot(t, comp, color='k', lw=1)
                axes[0].set_title(f'IC{ic:03d} source signal')
                axes[0].set_xlabel('Time (s)')
                axes[0].set_ylabel('Amplitude')
                axes[0].grid(alpha=0.3)

                # Channel weight barplot (signed)
                order = np.argsort(-np.abs(weights))
                axes[1].bar(np.array(ch_names)[order], weights[order],␣
↪color='tab:blue')
                axes[1].axhline(0, color='k', ls='--', lw=1)
                axes[1].set_title(f'IC{ic:03d} channel contribution weights')
                axes[1].set_ylabel('Mixing weight')
                axes[1].tick_params(axis='x', rotation=45)

                # Reconstructed per-channel contribution traces
                for c in range(recon.shape[0]):
                    axes[2].plot(t, recon[c] + c * np.std(recon) * 3, lw=1)
                axes[2].set_title(f'IC{ic:03d} reconstructed contribution per␣
↪channel')
                axes[2].set_xlabel('Time (s)')
                axes[2].set_ylabel('Contribution + offset')
                axes[2].grid(alpha=0.3)

                plt.suptitle(f'ICA decomposition diagnostics | {run_key} |␣
↪IC{ic:03d}', y=1.02)
                plt.tight_layout()
                plt.show()

                if 'Fz' in ch_names:
                    fz_idx = ch_names.index('Fz')
                    print(f"IC{ic:03d} Fz weight = {weights[fz_idx]:.4f}␣
↪(larger |value| => stronger frontal contribution)")

        if ICA_USE_MANUAL_SELECTION:
            print('Manual mode is ON. Edit ICA_MANUAL_EXCLUDE and rerun to set␣
↪exclusions.')

    # Apply exclusions
    if ICA_USE_MANUAL_SELECTION:
```

```
        exclude = ICA_MANUAL_EXCLUDE.get(run_key, suggested)
        if run_key not in ICA_MANUAL_EXCLUDE:
            print(f'No manual IC list found for {run_key}; using suggested:␣
↪{exclude}')
    else:
        exclude = suggested

    raw_clean = ica.apply(raw.copy(), exclude=exclude, verbose=False)
    eeg_clean = raw_clean.get_data()
    return eeg_clean, exclude


def make_filters(fs, bp=(1, 40), notch=60.0, order=4):
    nyq = fs / 2.0
    low, high = bp
    if high >= nyq:
        high = nyq - 1.0
    sos = butter(order, [low / nyq, high / nyq], btype='band', output='sos')

    # Notch only if feasible at this fs
    notch_ba = None
    if notch < nyq:
        b_notch, a_notch = iirnotch(w0=notch / nyq, Q=30)
        notch_ba = (b_notch, a_notch)
    return sos, notch_ba


def detect_bad_channels(epoch):
    """
    Detect bad channels on one epoch (n_channels, n_samples).
    Heuristics: flatline, extreme variance, extreme peak-to-peak.
    """
    ch_std = np.std(epoch, axis=1)
    ch_ptp = np.ptp(epoch, axis=1)

    med_std = np.median(ch_std) + 1e-12
    med_ptp = np.median(ch_ptp) + 1e-12

    flat = (ch_std < 0.05 * med_std) | (ch_ptp < 0.05 * med_ptp)
    noisy = (ch_std > 5.0 * med_std) | (ch_ptp > 6.0 * med_ptp)

    return np.where(flat | noisy)[0]


def preprocess_epoch(epoch, fs):
    """
```

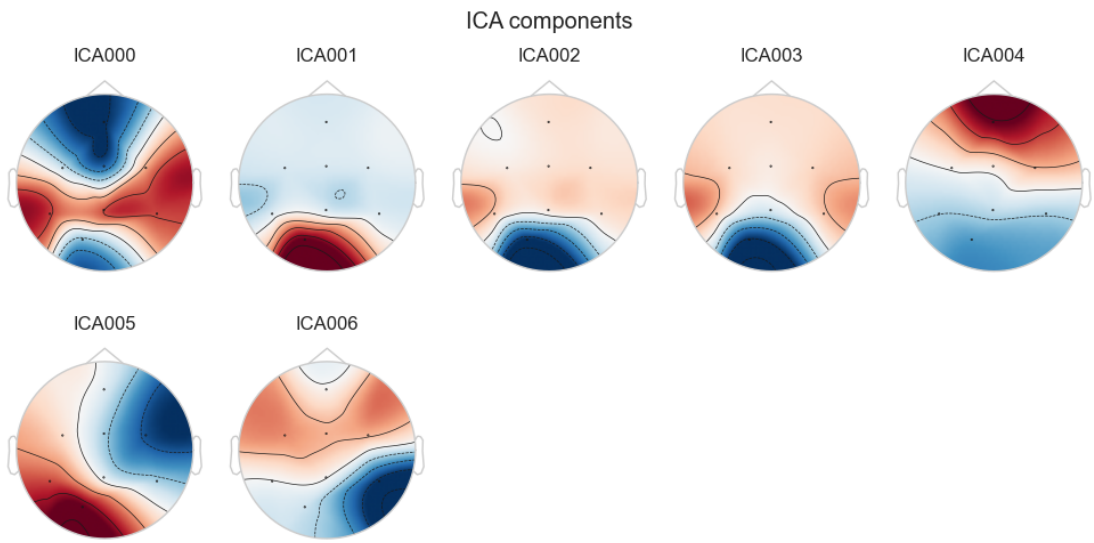
```
    Preprocess one epoch: detrend, notch, bandpass, CAR, baseline-correct,␣
↪bad-channel repair.
    """
    x = epoch.copy()

    # Detrend per channel
    x = detrend(x, axis=1, type='linear')

    # Filters
    sos, notch_ba = make_filters(fs, bp=(1, 40), notch=60.0, order=4)
    if notch_ba is not None:
        b_notch, a_notch = notch_ba
        x = filtfilt(b_notch, a_notch, x, axis=1)
    x = sosfiltfilt(sos, x, axis=1)

    # Common average reference (CAR)
    x = x - x.mean(axis=0, keepdims=True)

    # Baseline correction (first BASELINE_SEC inside epoch)
    b = max(1, int(BASELINE_SEC * fs))
    b = min(b, x.shape[1])
    x = x - x[:, :b].mean(axis=1, keepdims=True)

    # Bad channel detection + repair by median of good channels
    bad_idx = detect_bad_channels(x)
    if len(bad_idx) > 0:
        good_idx = np.array([i for i in range(x.shape[0]) if i not in bad_idx])
        if len(good_idx) == 0:
            return None, bad_idx
        replacement = np.median(x[good_idx], axis=0, keepdims=True)
        x[bad_idx] = replacement

    return x, bad_idx


def bandpowers_welch(epoch, fs, bands=BANDS):
    """Return dict band -> array(n_channels,) of power by integrating Welch PSD.
↪"""
    nperseg = min(epoch.shape[1], int(fs * 1.0))
    freqs, psd = welch(epoch, fs=fs, axis=1, nperseg=nperseg)

    out = {}
    for band_name, (f0, f1) in bands.items():
        mask = (freqs >= f0) & (freqs < f1)
        if not np.any(mask):
            out[band_name] = np.zeros(epoch.shape[0])
        else:
```

```
            out[band_name] = np.trapezoid(psd[:, mask], freqs[mask], axis=1)
    return out
```

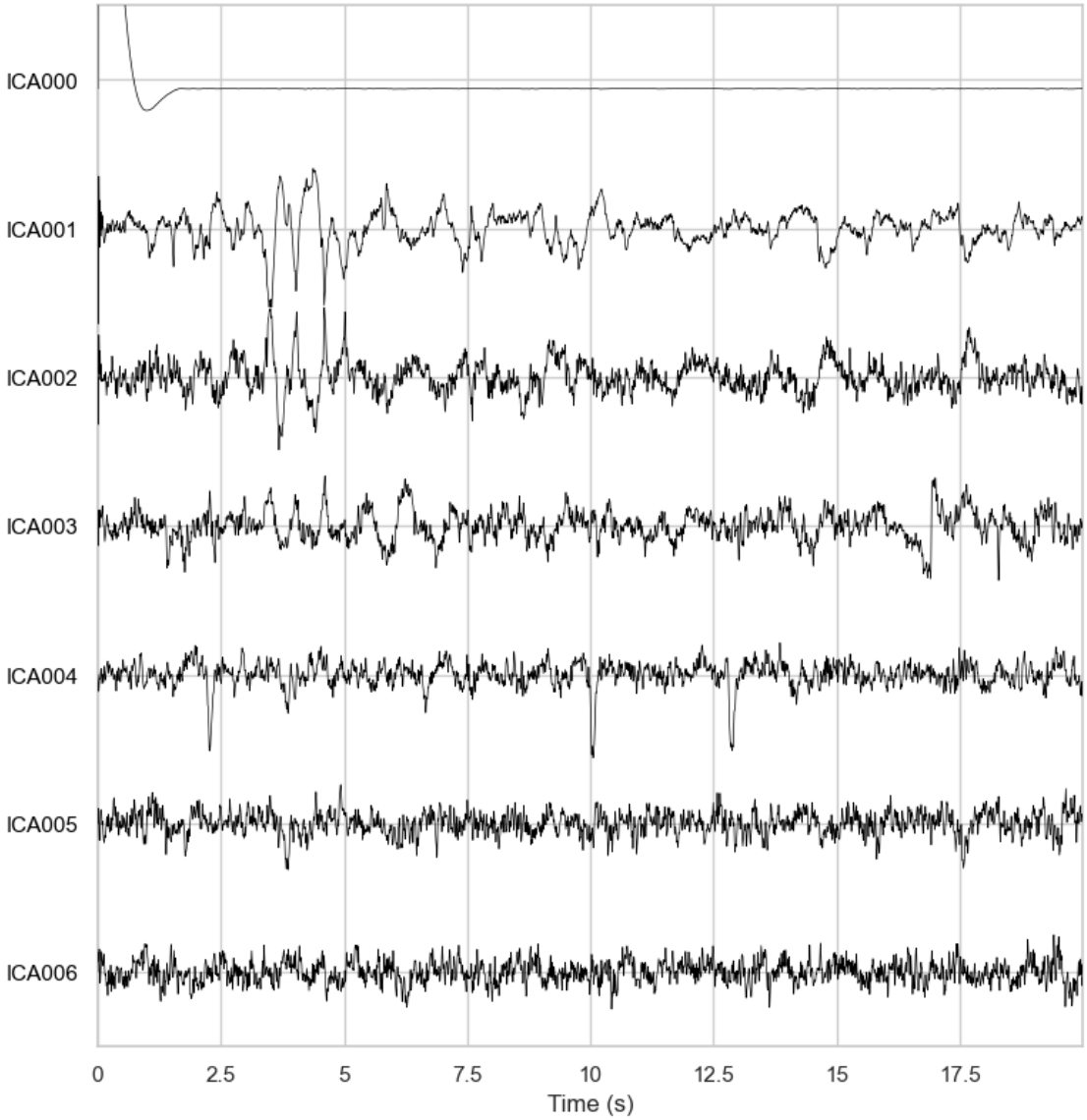
[4]:
```
# Build trial-level dataset (AI/Human × Neutral/Happy only)
all_epochs = []
all_labels = []
all_meta = []
bad_counter = Counter()
ica_exclude_log = []

for _, row in run_df.iterrows():
    run_path = row['run_path']
    eeg = np.load(os.path.join(run_path, 'eeg.npy'))
    ts = np.load(os.path.join(run_path, 'timestamp.npy'))
    with open(os.path.join(run_path, 'markers.json'), 'r') as f:
        markers = json.load(f)

    eeg = ensure_ch_by_time(eeg)  # (channels, samples)

    fs_run = TARGET_FS
    L_run = TARGET_N_SAMPLES

    # Run-level ICA cleaning (extended Infomax) with visualization + manual␣
↪selection
    run_ch_names = CH_NAMES[:eeg.shape[0]]
    run_key = f"{row['subject']}/{row['session']}/{row['run']}"
    eeg, excluded_ics = run_extended_infomax_ica(eeg, fs_run, run_ch_names,␣
↪run_key=run_key)
    ica_exclude_log.append({
        'subject': row['subject'],
        'session': row['session'],
        'run': row['run'],
        'excluded_ics': excluded_ics,
    })

    for m in markers:
        cat = str(m.get('category', '')).strip()
        scn = str(m.get('scenario', '')).strip()

        if cat not in KEEP_CATEGORIES or scn not in KEEP_SCENARIOS:
            continue

        a = int(m['start_sample_index'])
        b = a + L_run
        if a < 0 or b > eeg.shape[1]:
            continue
```

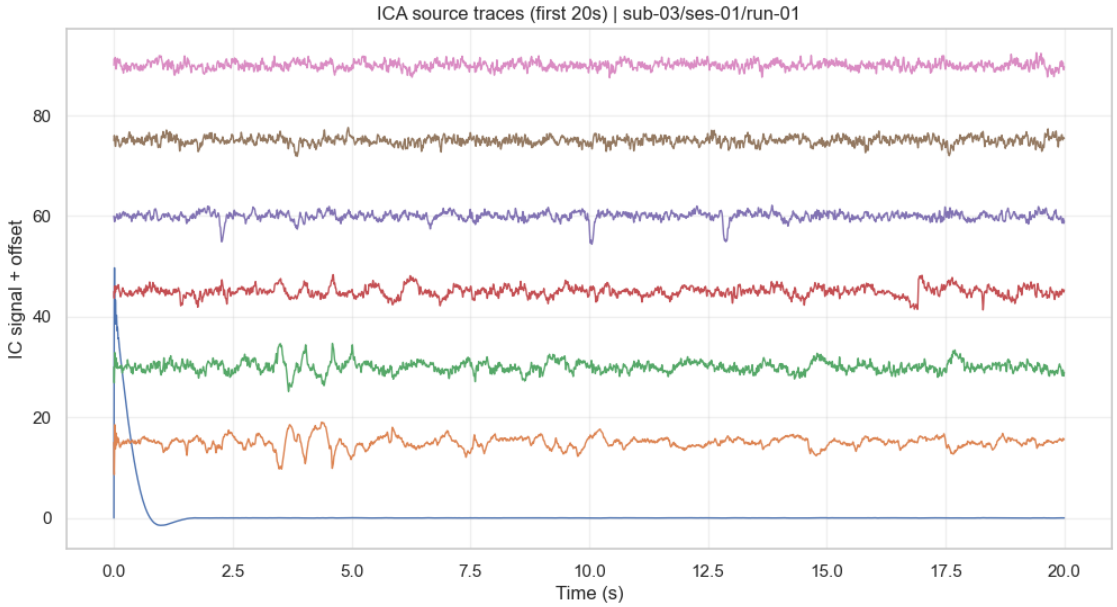
```
        epoch = eeg[:, a:b]  # fixed-length window at 250 Hz
        clean_epoch, bad_idx = preprocess_epoch(epoch, fs_run)
        if clean_epoch is None:
            continue

        # Safety check: keep a common shape for downstream modeling
        if clean_epoch.shape[1] != TARGET_N_SAMPLES:
            clean_epoch = resample(clean_epoch, TARGET_N_SAMPLES, axis=1)

        for idx in bad_idx:
            if idx < len(CH_NAMES):
                bad_counter[CH_NAMES[idx]] += 1
            else:
                bad_counter[f'ch{idx}'] += 1

        all_epochs.append(clean_epoch)
        all_labels.append(1 if cat == 'AI' else 0)
        all_meta.append({
            'subject': row['subject'],
            'session': row['session'],
            'run': row['run'],
            'category': cat,
            'scenario': scn,
            'fs_run': fs_run,
            'trial_index': m.get('trial_index', None),
        })

X_epochs = np.stack(all_epochs) if all_epochs else np.empty((0, len(CH_NAMES),␣
 ↪TARGET_N_SAMPLES))
y = np.array(all_labels, dtype=int)
meta_df = pd.DataFrame(all_meta)
ica_df = pd.DataFrame(ica_exclude_log)

print('Epoch tensor shape:', X_epochs.shape, '(n_trials, n_channels, n_time)')
print('Class counts (0=Human, 1=AI):', Counter(y))
if len(meta_df):
    display(pd.crosstab(meta_df['category'], meta_df['scenario']))

print('\nSampling rate used in analysis:', TARGET_FS, 'Hz')
print('ICA exclusions per run (extended Infomax):')
if len(ica_df):
    display(ica_df)
else:
    print('No ICA logs available.')

print('Bad-channel repair counts across accepted trials:')
print(dict(bad_counter))
```

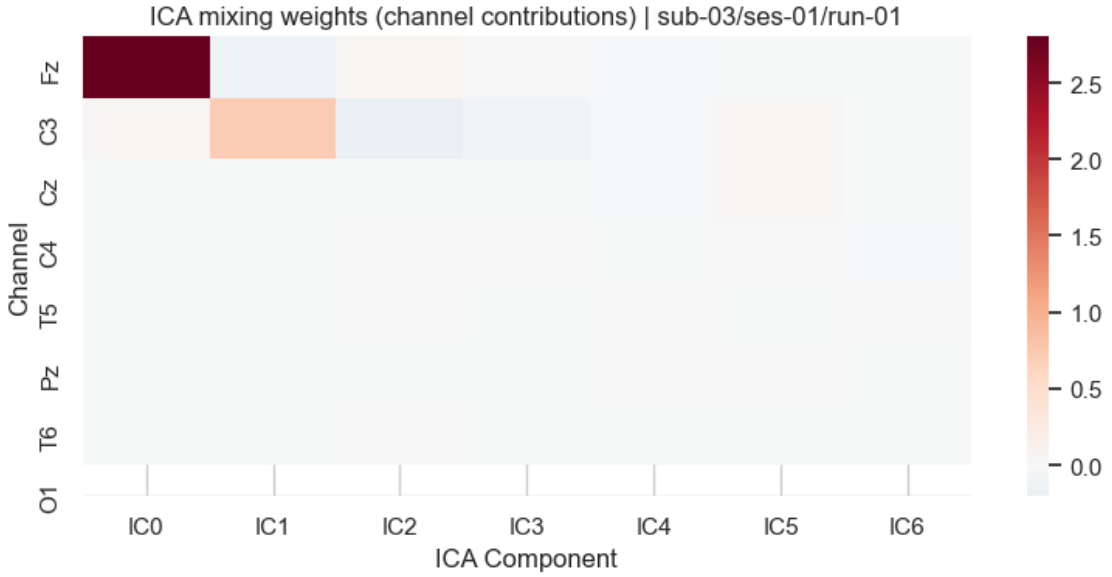
```
=== ICA review: sub-03/ses-01/run-01 ===
Suggested artifact ICs (kurtosis heuristic): [0]
```

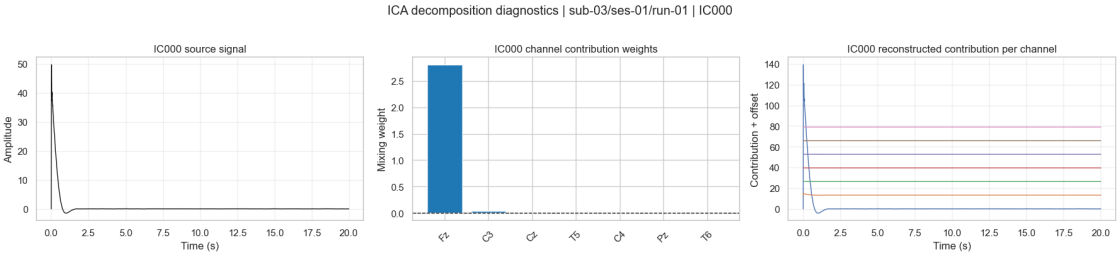



```
<Figure size 640x480 with 0 Axes>

Creating RawArray with float64 data, n_channels=7, n_times=89701
    Range : 0 … 89700 =      0.000 …   358.800 secs
Ready.
Using matplotlib as 2D backend.
```

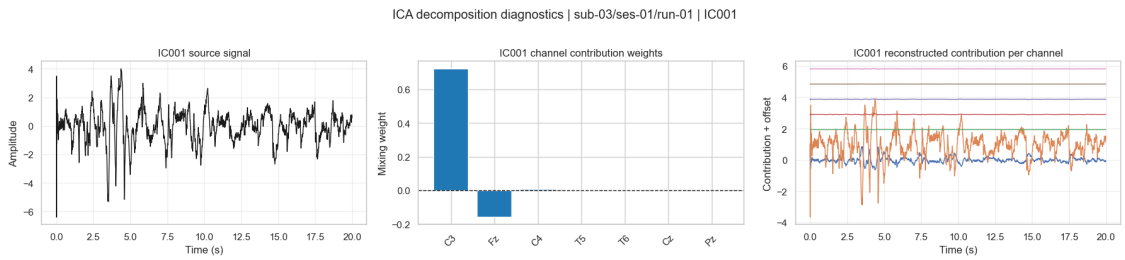

ICA000
ICA001
ICA002
ICA003
ICA004
ICA005
ICA006
0
2.5
5
7.5
10
12.5
15
17.5
Time (s)


ICA source traces (first 20s) | sub-03/ses-01/run-01
IC signal + offset
0
20
40
60
80
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Time (s)

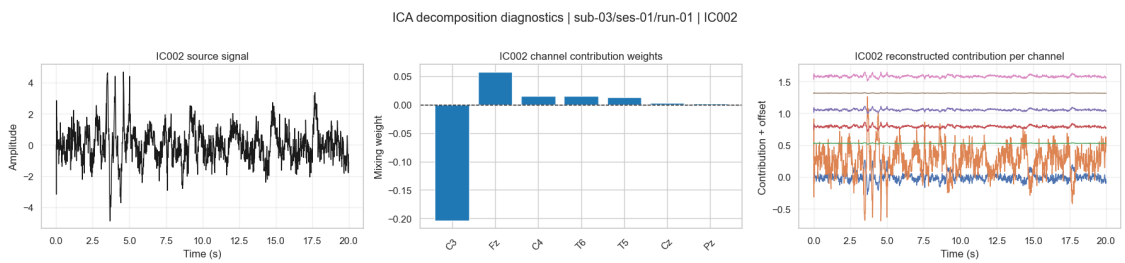

ICA mixing weights (channel contributions) | sub-03/ses-01/run-01
Channel
Fz
C3
Cz
C4
T5
Pz
T6
O1
IC0
IC1
IC2
IC3
IC4
IC5
IC6
ICA Component
0.0
0.5
1.0
1.5
2.0
2.5


ICA decomposition diagnostics | sub-03/ses-01/run-01 | IC000
IC000 source signal
Amplitude
Time (s)
IC000 channel contribution weights
Mixing weight
Fz
C3
Cz
T5
C4
Pz
T6
IC000 reconstructed contribution per channel
Contribution + offset
Time (s)

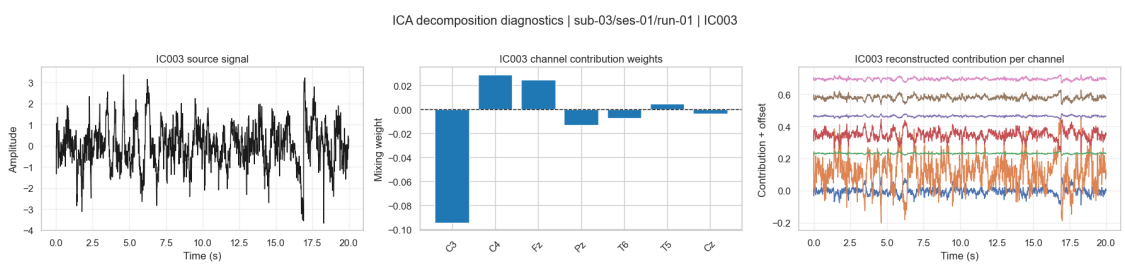
IC000 Fz weight = 2.8021 (larger |value| => stronger frontal contribution)



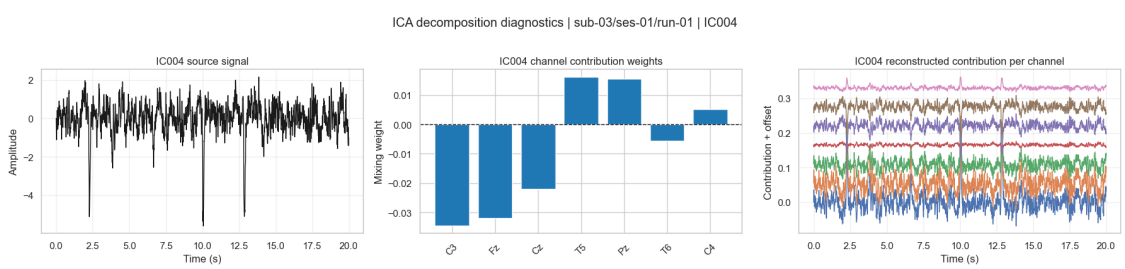
IC001 Fz weight = -0.1594 (larger |value| => stronger frontal contribution)



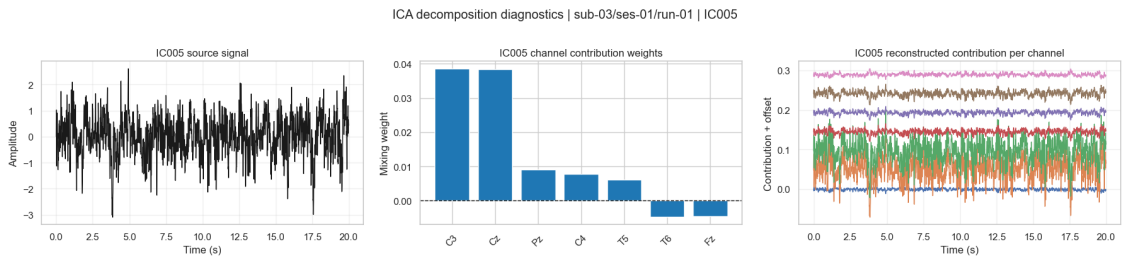
IC002 Fz weight = 0.0573 (larger |value| => stronger frontal contribution)



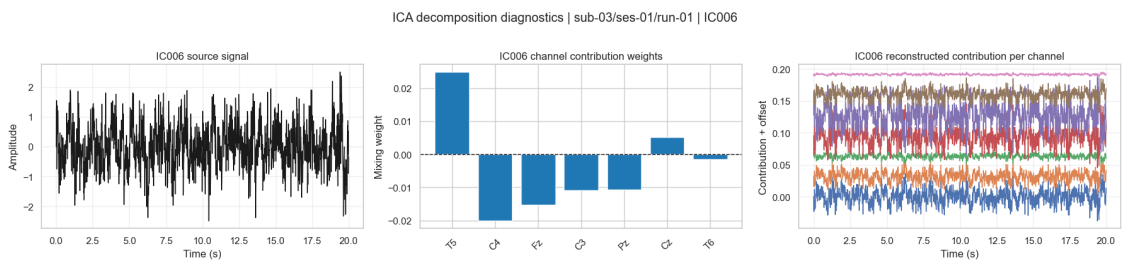
IC003 Fz weight = 0.0249 (larger |value| => stronger frontal contribution)



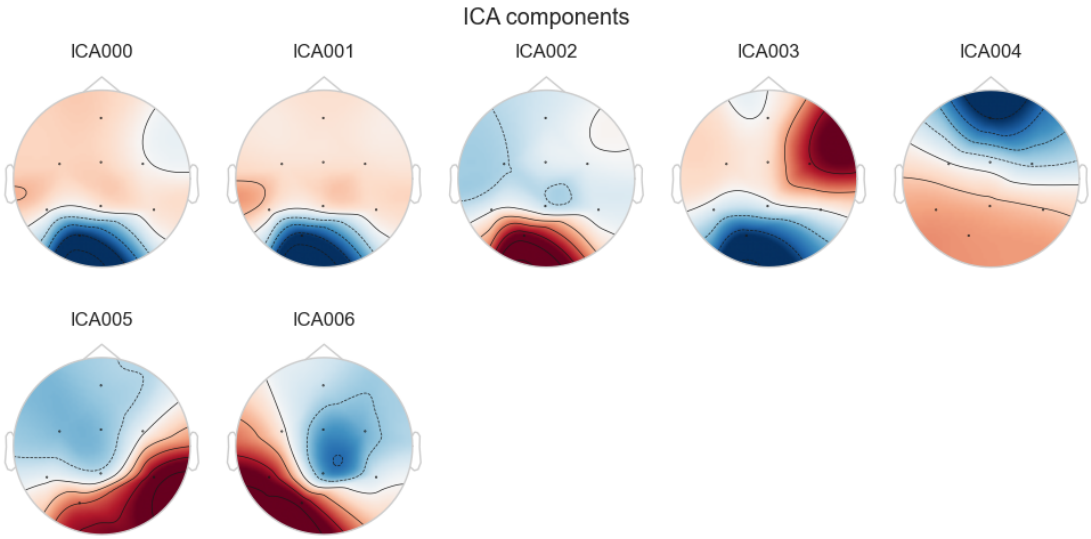
```
IC004 Fz weight = -0.0320 (larger |value| => stronger frontal contribution)
```



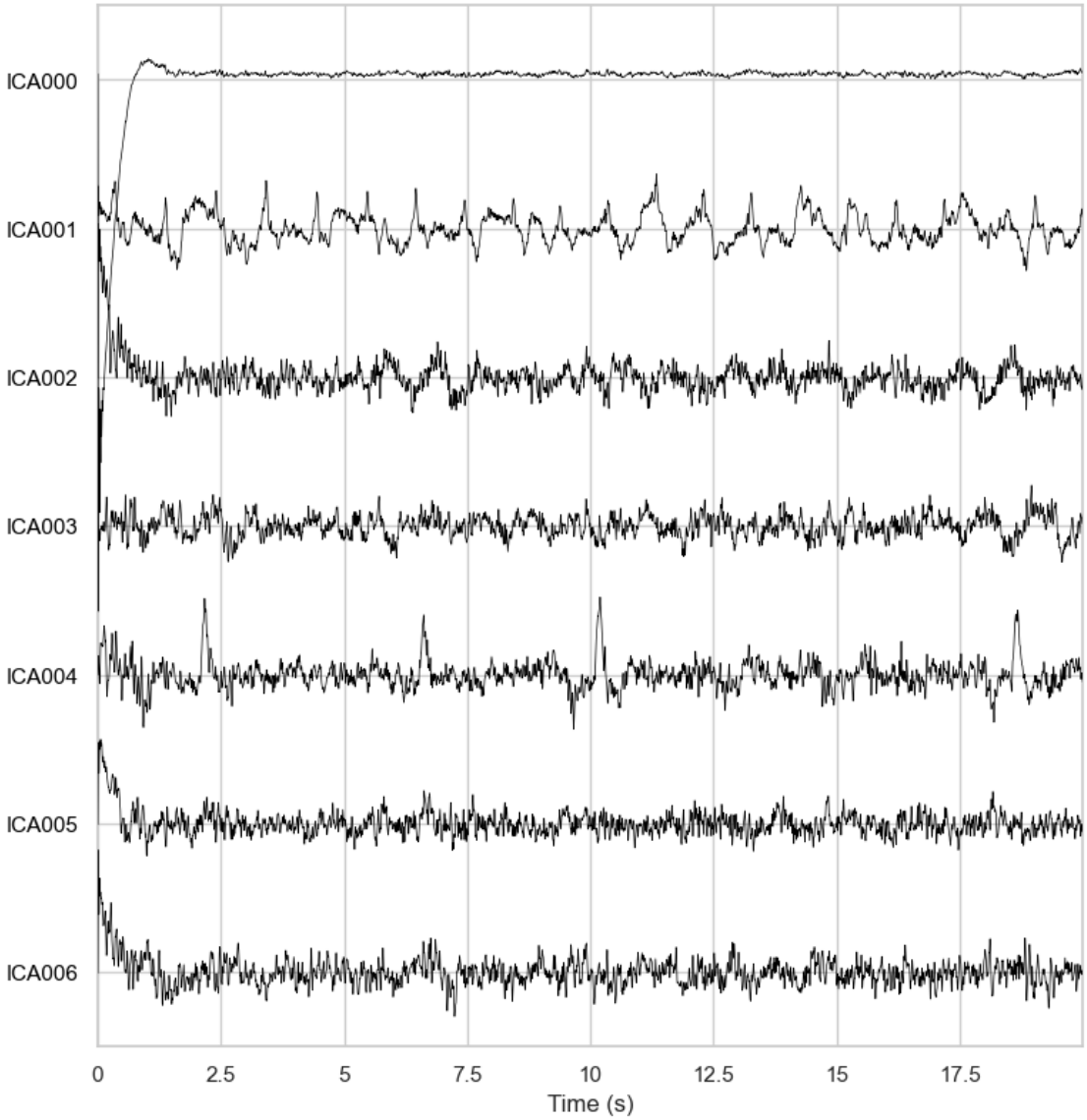

```
IC005 Fz weight = -0.0045 (larger |value| => stronger frontal contribution)
```

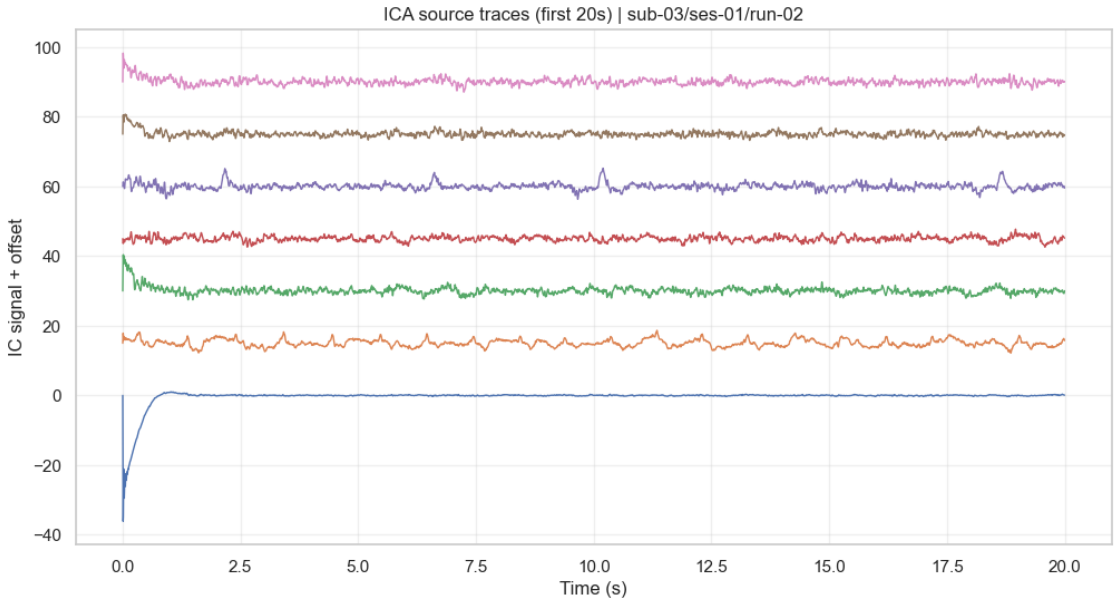


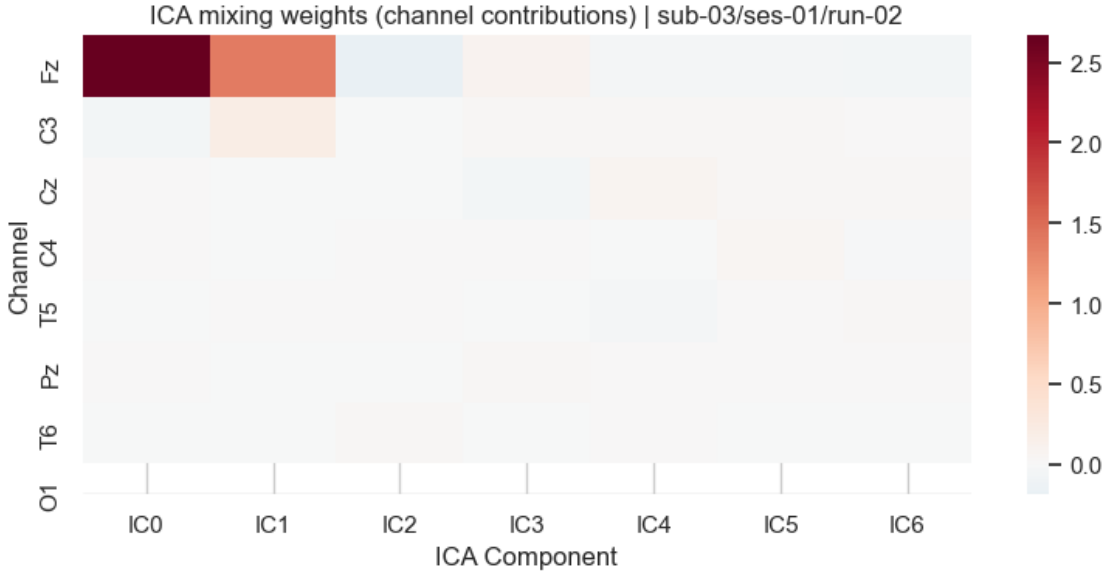

```
IC006 Fz weight = -0.0152 (larger |value| => stronger frontal contribution)
Manual mode is ON. Edit ICA_MANUAL_EXCLUDE and rerun to set exclusions.
No manual IC list found for sub-03/ses-01/run-01; using suggested: [0]

=== ICA review: sub-03/ses-01/run-02 ===
Suggested artifact ICs (kurtosis heuristic): [0]
```

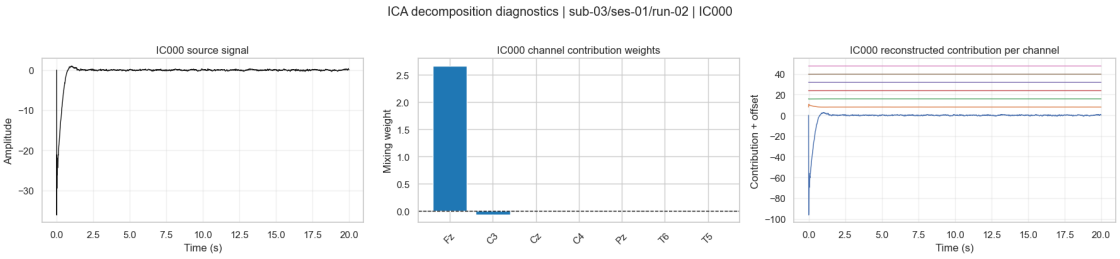



```
<Figure size 640x480 with 0 Axes>

Creating RawArray with float64 data, n_channels=7, n_times=89581
    Range : 0 … 89580 =      0.000 …   358.320 secs
Ready.
```

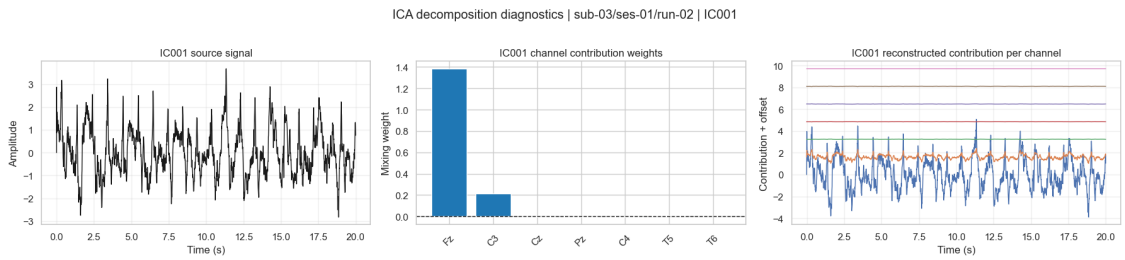

ICA000
ICA001
ICA002
ICA003
ICA004
ICA005
ICA006
0
2.5
5
7.5
10
12.5
15
17.5
Time (s)


ICA source traces (first 20s) | sub-03/ses-01/run-02
IC signal + offset
100
80
60
40
20
0
−20
−40
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Time (s)

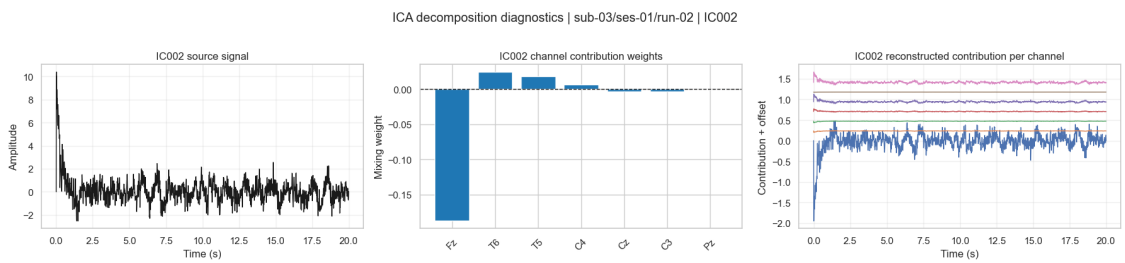

ICA mixing weights (channel contributions) | sub-03/ses-01/run-02
Channel
Fz
C3
Cz
C4
T5
Pz
T6
O1
IC0
IC1
IC2
IC3
IC4
IC5
IC6
ICA Component
2.5
2.0
1.5
1.0
0.5
0.0


ICA decomposition diagnostics | sub-03/ses-01/run-02 | IC000
IC000 source signal
Amplitude
Time (s)
IC000 channel contribution weights
Mixing weight
Fz
C3
Cz
C4
Pz
T6
T5
IC000 reconstructed contribution per channel
Contribution + offset
Time (s)

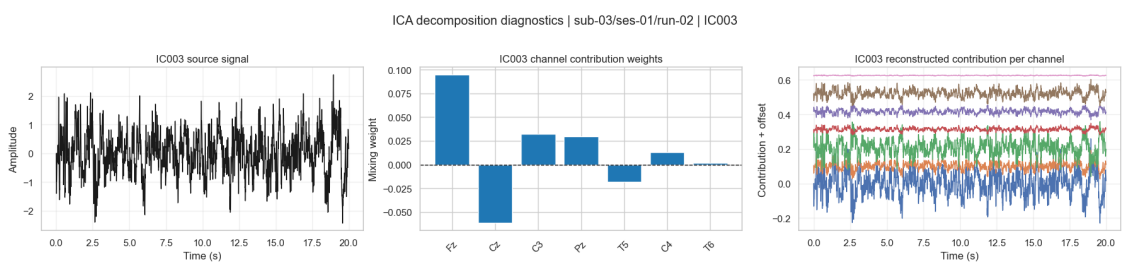
IC000 Fz weight = 2.6656 (larger |value| => stronger frontal contribution)



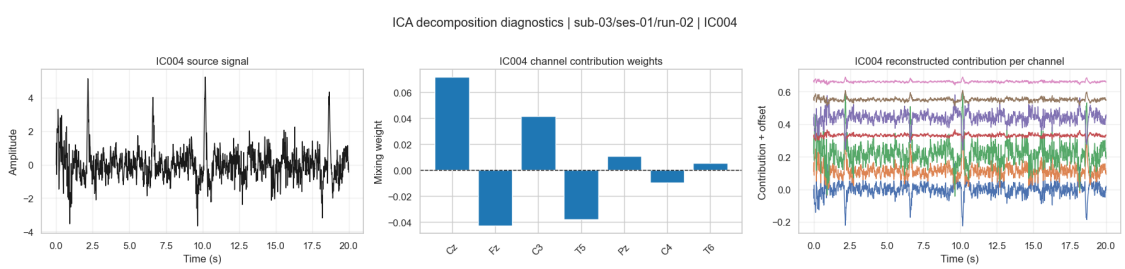
IC001 Fz weight = 1.3840 (larger |value| => stronger frontal contribution)



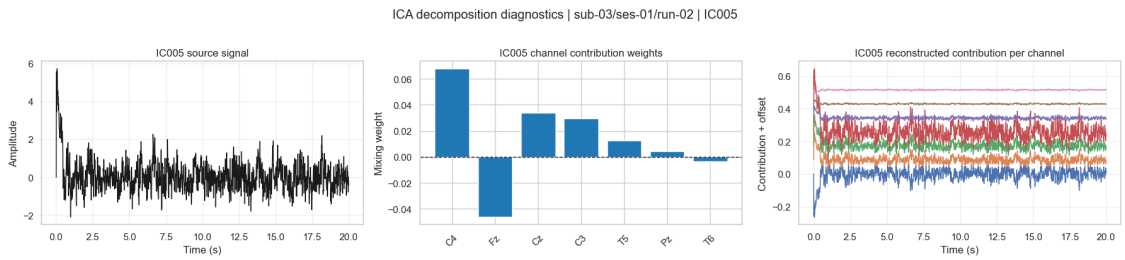
IC002 Fz weight = -0.1875 (larger |value| => stronger frontal contribution)



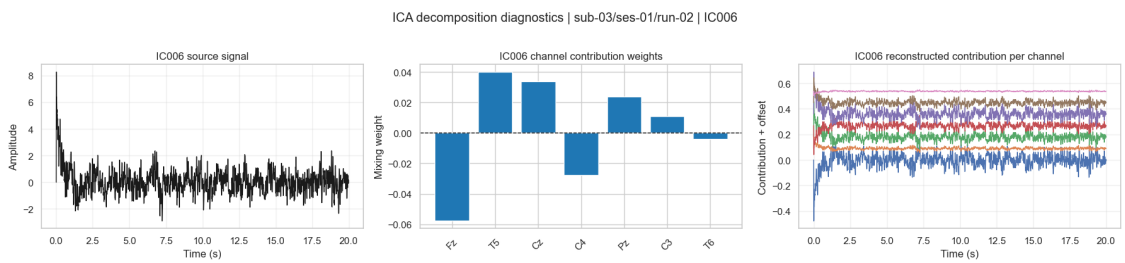
IC003 Fz weight = 0.0946 (larger |value| => stronger frontal contribution)



```
IC004 Fz weight = -0.0431 (larger |value| => stronger frontal contribution)
```




```
IC005 Fz weight = -0.0463 (larger |value| => stronger frontal contribution)
```



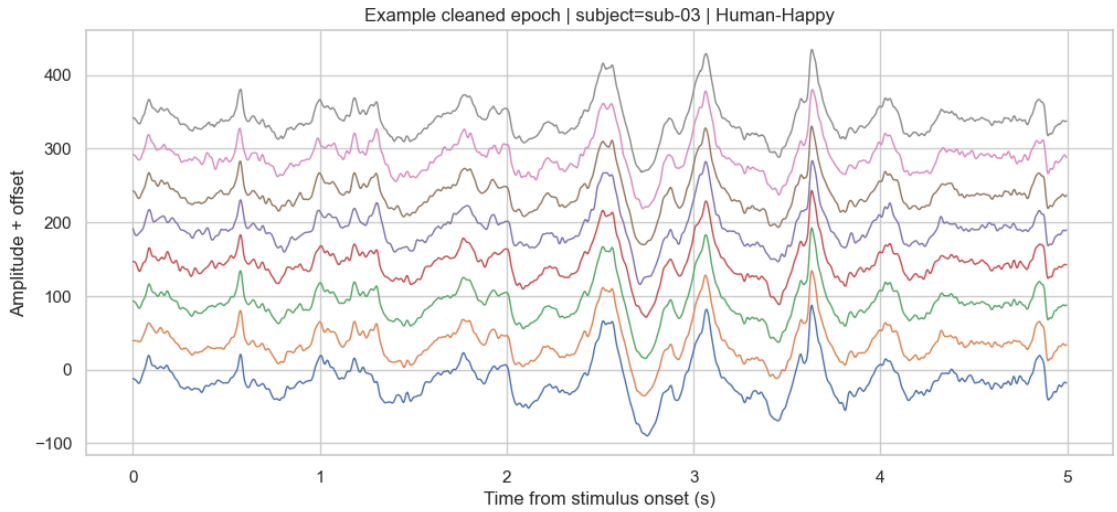

```
IC006 Fz weight = -0.0578 (larger |value| => stronger frontal contribution)
Manual mode is ON. Edit ICA_MANUAL_EXCLUDE and rerun to set exclusions.
No manual IC list found for sub-03/ses-01/run-02; using suggested: [0]
Epoch tensor shape: (120, 8, 1250) (n_trials, n_channels, n_time)
Class counts (0=Human, 1=AI): Counter({np.int64(0): 60, np.int64(1): 60})

scenario  Happy
category
AI           60
Human        60


Sampling rate used in analysis: 250.0 Hz
ICA exclusions per run (extended Infomax):

  subject session     run excluded_ics
0  sub-03  ses-01  run-01          [0]
1  sub-03  ses-01  run-02          [0]

Bad-channel repair counts across accepted trials:
{'O1': 113}
```

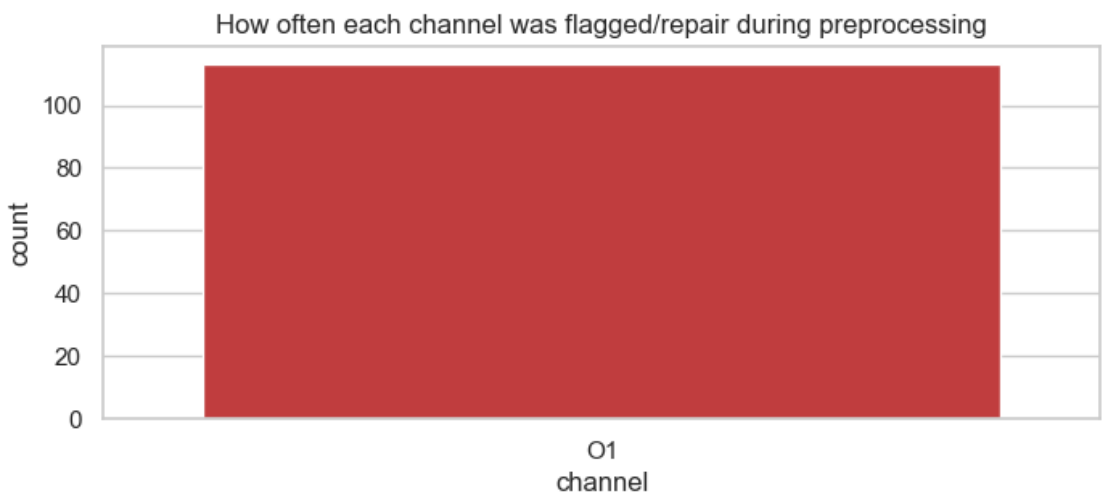
[5]:
```
# Split into development set and independent final test set
if len(y) == 0:
    raise RuntimeError('No labels found. Run data build cell first.')

idx_all = np.arange(len(y))
train_idx, final_idx = train_test_split(
    idx_all,
    test_size=FINAL_TEST_SIZE,
    stratify=y,
    random_state=RANDOM_STATE,
    shuffle=True,
)

print(f'Development size: {len(train_idx)} | Final independent test size:␣
↪{len(final_idx)}')
print('Class counts in development set:', Counter(y[train_idx]))
print('Class counts in final test set:', Counter(y[final_idx]))
```

```
Development size: 96 | Final independent test size: 24
Class counts in development set: Counter({np.int64(1): 48, np.int64(0): 48})
Class counts in final test set: Counter({np.int64(1): 12, np.int64(0): 12})
```

[5]:
```
# Quick visual QA: example trial and channel traces
if len(X_epochs) > 0:
    i = 0
    fs0 = TARGET_FS
    t = np.arange(X_epochs.shape[-1]) / fs0

    fig, ax = plt.subplots(figsize=(12, 5))
    epoch0 = X_epochs[i]
    for ch in range(epoch0.shape[0]):
        ax.plot(t, epoch0[ch] + ch * 50, lw=1)
    ax.set_title(f"Example cleaned epoch | subject={meta_df.iloc[i]['subject']}␣
↪| {meta_df.iloc[i]['category']}-{meta_df.iloc[i]['scenario']}")
    ax.set_xlabel('Time from stimulus onset (s)')
    ax.set_ylabel('Amplitude + offset')
    plt.show()

# Bar plot of bad-channel frequency
if len(bad_counter):
    bad_df = pd.DataFrame({'channel': list(bad_counter.keys()), 'count':␣
↪list(bad_counter.values())})
    bad_df = bad_df.sort_values('count', ascending=False)

    plt.figure(figsize=(8, 3))
    sns.barplot(data=bad_df, x='channel', y='count', color='tab:red')
    plt.title('How often each channel was flagged/repair during preprocessing')
```

```
    plt.show()
```



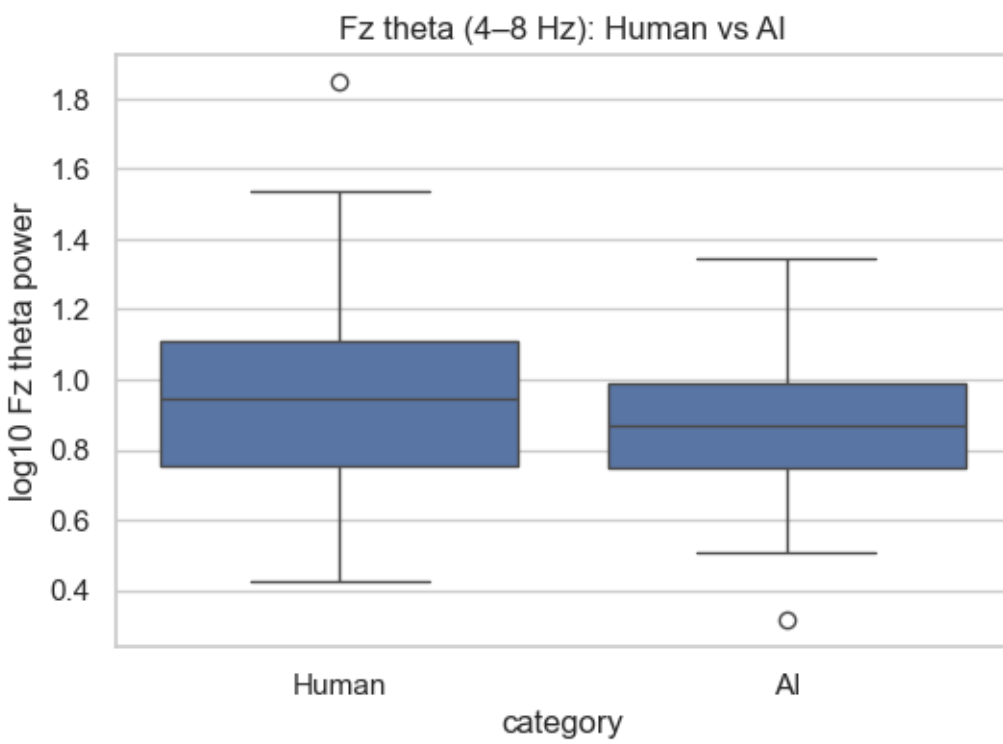


[6]:
```
# Feature extraction: channel-wise bandpowers
feature_rows = []

for i in range(len(X_epochs)):
    bp = bandpowers_welch(X_epochs[i], fs=TARGET_FS, bands=BANDS)

    row = {
        'y': y[i],
        'category': meta_df.iloc[i]['category'],
        'scenario': meta_df.iloc[i]['scenario'],
```

```
        'subject': meta_df.iloc[i]['subject'],
    }

    for ch_i, ch in enumerate(CH_NAMES):
        for b in BANDS.keys():
            row[f'{ch}_{b}'] = bp[b][ch_i]

    feature_rows.append(row)

feat_df = pd.DataFrame(feature_rows)
print('Feature table shape:', feat_df.shape)
display(feat_df.head(3))

# Frontal theta hypothesis check at Fz
fz_theta_h = feat_df.loc[feat_df['category'] == 'Human', 'Fz_theta'].to_numpy()
fz_theta_ai = feat_df.loc[feat_df['category'] == 'AI', 'Fz_theta'].to_numpy()

# Log-transform for better Gaussian behavior
fz_theta_h_log = np.log10(fz_theta_h + 1e-12)
fz_theta_ai_log = np.log10(fz_theta_ai + 1e-12)

stat = ttest_ind(fz_theta_ai_log, fz_theta_h_log, equal_var=False)

print('\nFz theta (log10 power)')
print(f'Human mean ± sd: {fz_theta_h_log.mean():.4f} ± {fz_theta_h_log.std():.
 ↪4f}  (n={len(fz_theta_h_log)})')
print(f'AI    mean ± sd: {fz_theta_ai_log.mean():.4f} ± {fz_theta_ai_log.std():.
 ↪4f}  (n={len(fz_theta_ai_log)})')
print(f'Welch t-test: t={stat.statistic:.3f}, p={stat.pvalue:.4g}')

plt.figure(figsize=(6, 4))
sns.boxplot(data=feat_df, x='category', y=np.log10(feat_df['Fz_theta'] +␣
 ↪1e-12), order=['Human', 'AI'])
plt.ylabel('log10 Fz theta power')
plt.title('Fz theta (4-8 Hz): Human vs AI')
plt.show()
```

```
Feature table shape: (120, 44)

   y category scenario subject    Fz_delta   Fz_theta   Fz_alpha   Fz_beta  \
0  0    Human    Happy  sub-03  532.241914  70.361793  12.338785  9.375659
1  1       AI    Happy  sub-03  110.162645  19.912153   4.970367  4.315487
2  0    Human    Happy  sub-03   47.236573  12.381894   2.265410  2.775625

   Fz_gamma    C3_delta  …    T6_delta   T6_theta   T6_alpha   T6_beta  \
0  0.775492  484.579339  …  445.500693  66.292273  12.494981  8.015944
1  0.639668   81.011859  …   67.550231  13.186725   3.568824  5.023978
2  0.542124   39.329382  …   48.581157   7.614749   3.689198  3.762709
```

```
   T6_gamma    O1_delta   O1_theta   O1_alpha   O1_beta  O1_gamma
0  0.962756  464.881121  65.952427  11.030626  7.238404  0.509865
1  0.798025   79.072266  14.655944   3.381492  2.704768  0.360586
2  0.714994   38.658826   6.775002   2.250730  1.476287  0.279294

[3 rows x 44 columns]


Fz theta (log10 power)
Human mean ± sd: 0.9498 ± 0.2560  (n=60)
AI    mean ± sd: 0.8680 ± 0.2153  (n=60)
Welch t-test: t=-1.879, p=0.06275
```



[10]:
```python
# Time-window scan for best classification range (development data only)
# Uses log-variance features per channel over sliding windows.

if 'train_idx' not in globals() or 'final_idx' not in globals():
    idx_all = np.arange(len(y))
    train_idx, final_idx = train_test_split(
        idx_all, test_size=FINAL_TEST_SIZE, stratify=y,␣
 ↪random_state=RANDOM_STATE, shuffle=True
    )
```

```
WINDOW_SEC = 0.30
WINDOW_STEP_SEC = 0.02  # 20 ms step

win_len = int(round(WINDOW_SEC * TARGET_FS))
step_len = max(1, int(round(WINDOW_STEP_SEC * TARGET_FS)))

# Development-only data for model selection
X_dev_epochs = X_epochs[train_idx]
y_dev = y[train_idx]

n_t = X_dev_epochs.shape[-1]
start_idx = np.arange(0, max(1, n_t - win_len + 1), step_len)
starts_sec = start_idx / TARGET_FS

skf_dev = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

scan_scores = []
for s in start_idx:
    seg = X_dev_epochs[:, :, s:s + win_len]

    # Feature = log-variance per channel in the window
    Xw = np.log10(np.var(seg, axis=2) + 1e-12)

    score = cross_val_score(
        Pipeline([
            ('scaler', StandardScaler()),
            ('clf', LinearDiscriminantAnalysis()),
        ]),
        Xw,
        y_dev,
        cv=skf_dev,
        scoring='accuracy',
    ).mean()
    scan_scores.append(score)

scan_scores = np.array(scan_scores)
best_i = int(np.argmax(scan_scores))
best_start_sec = float(starts_sec[best_i])
best_end_sec = float(best_start_sec + WINDOW_SEC)
best_acc_dev = float(scan_scores[best_i])

plt.figure(figsize=(10, 3.5))
plt.plot(starts_sec, scan_scores, lw=2)
plt.axvline(best_start_sec, ls='--', c='tab:red', label=f'best␣
 ↪start={best_start_sec:.3f}s')
plt.xlabel('Window start (s)')
```

```
plt.ylabel('CV accuracy (development only)')
plt.title(f'Time-window scan on development set (win={WINDOW_SEC:.2f}s)')
plt.grid(alpha=0.3)
plt.legend()
plt.show()

# Evaluate selected window once on independent final test set
Xw_dev_best = np.log10(np.var(X_dev_epochs[:, :, start_idx[best_i]:
 ↪start_idx[best_i] + win_len], axis=2) + 1e-12)
Xw_final_best = np.log10(np.var(X_epochs[final_idx][:, :, start_idx[best_i]:
 ↪start_idx[best_i] + win_len], axis=2) + 1e-12)
y_final = y[final_idx]

pipe_tw = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LinearDiscriminantAnalysis()),
])
pipe_tw.fit(Xw_dev_best, y_dev)
y_pred_final = pipe_tw.predict(Xw_final_best)
y_score_final = pipe_tw.predict_proba(Xw_final_best)[:, 1]

print(f'Best time window (selected on development): {best_start_sec:.3f}s to␣
 ↪{best_end_sec:.3f}s')
print(f'Best window development CV accuracy: {best_acc_dev:.3f}')
print('Final independent test metrics for selected window:')
print(f"  acc={accuracy_score(y_final, y_pred_final):.3f},␣
 ↪auc={roc_auc_score(y_final, y_score_final):.3f}, f1={f1_score(y_final,␣
 ↪y_pred_final):.3f}")
```

```
Best time window (selected on development): 2.020s to 2.320s
Best window development CV accuracy: 0.667
```

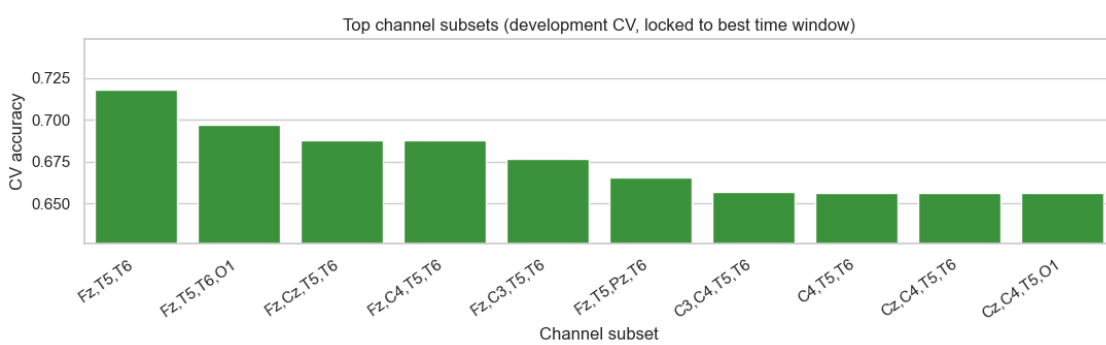
```
Final independent test metrics for selected window:
  acc=0.542, auc=0.542, f1=0.560
```

[15]:
```python
# Channel subset selection locked to the best time window (development only)
# Then evaluate selected subset once on independent final test.

from itertools import combinations

if 'best_i' not in globals() or 'start_idx' not in globals() or 'win_len' not␣
↪in globals():
    raise RuntimeError('Run the time-window scan cell first to define best␣
↪window.')
if 'train_idx' not in globals() or 'final_idx' not in globals():
    raise RuntimeError('Run the split cell first to define development/final␣
↪indices.')

# Build best-window features for all channels (log-variance)
s0 = int(start_idx[best_i])
Xw_dev_all = np.log10(np.var(X_epochs[train_idx][:, :, s0:s0 + win_len],␣
↪axis=2) + 1e-12)
Xw_final_all = np.log10(np.var(X_epochs[final_idx][:, :, s0:s0 + win_len],␣
↪axis=2) + 1e-12)
y_dev = y[train_idx]
y_final = y[final_idx]

skf_subset = StratifiedKFold(n_splits=5, shuffle=True,␣
↪random_state=RANDOM_STATE)

subset_rows = []
best_subset = None
best_score = -np.inf

# Search subset sizes 1..4 to avoid overfitting on small sample size
max_k = min(4, len(CH_NAMES))
for k in range(1, max_k + 1):
    for combo in combinations(range(len(CH_NAMES)), k):
        X_dev_sub = Xw_dev_all[:, combo]
        score = cross_val_score(
            Pipeline([
                ('scaler', StandardScaler()),
                ('clf', LinearDiscriminantAnalysis()),
            ]),
            X_dev_sub,
            y_dev,
            cv=skf_subset,
            scoring='accuracy',
        ).mean()
```

```
        subset_rows.append({
            'n_channels': k,
            'channels': ','.join([CH_NAMES[i] for i in combo]),
            'dev_cv_acc': score,
        })

        if score > best_score:
            best_score = score
            best_subset = combo

subset_df = pd.DataFrame(subset_rows).sort_values(['dev_cv_acc', 'n_channels'],␣
 ↪ascending=[False, True])
display(subset_df.head(15))

# Final independent evaluation for best subset
X_dev_bestch = Xw_dev_all[:, best_subset]
X_final_bestch = Xw_final_all[:, best_subset]

pipe_ch = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LinearDiscriminantAnalysis()),
])
pipe_ch.fit(X_dev_bestch, y_dev)
y_pred_final_ch = pipe_ch.predict(X_final_bestch)
y_score_final_ch = pipe_ch.predict_proba(X_final_bestch)[:, 1]

best_channels = [CH_NAMES[i] for i in best_subset]
print('Best channel subset (selected on development CV, locked to best window):
 ↪', best_channels)
print(f'Development CV accuracy (best subset): {best_score:.3f}')
print('Final independent test metrics (best subset @ best window):')
print(f"  acc={accuracy_score(y_final, y_pred_final_ch):.3f},␣
 ↪auc={roc_auc_score(y_final, y_score_final_ch):.3f}, f1={f1_score(y_final,␣
 ↪y_pred_final_ch):.3f}")

# Quick plot of top subsets
top_plot = subset_df.head(10).copy()
plt.figure(figsize=(11, 3.5))
sns.barplot(data=top_plot, x='channels', y='dev_cv_acc', color='tab:green')
plt.xticks(rotation=35, ha='right')
plt.ylim(max(0.3, top_plot['dev_cv_acc'].min() - 0.03), min(1.0,␣
 ↪top_plot['dev_cv_acc'].max() + 0.03))
plt.title('Top channel subsets (development CV, locked to best time window)')
plt.ylabel('CV accuracy')
plt.xlabel('Channel subset')
plt.tight_layout()
```

```
plt.show()
```

```
     n_channels     channels  dev_cv_acc
52            3     Fz,T5,T6    0.717895
125           4  Fz,T5,T6,O1    0.696842
112           4  Fz,Cz,T5,T6    0.687895
118           4  Fz,C4,T5,T6    0.687895
102           4  Fz,C3,T5,T6    0.676842
123           4  Fz,T5,Pz,T6    0.665789
138           4  C3,C4,T5,T6    0.656842
83            3     C4,T5,T6    0.656316
148           4  Cz,C4,T5,T6    0.656316
149           4  Cz,C4,T5,O1    0.656316
153           4  Cz,T5,Pz,T6    0.656316
159           4  C4,T5,T6,O1    0.656316
29            2        C4,O1    0.646842
27            2        C4,Pz    0.645789
43            3     Fz,Cz,T5    0.645789

Best channel subset (selected on development CV, locked to best window): ['Fz',
'T5', 'T6']
Development CV accuracy (best subset): 0.718
Final independent test metrics (best subset @ best window):
  acc=0.583, auc=0.535, f1=0.583
```

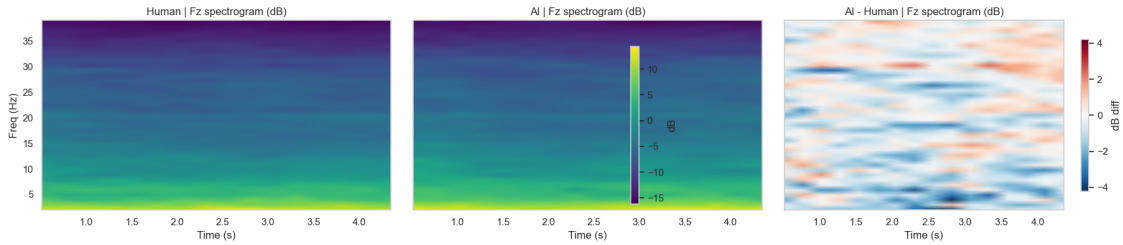


[23]:
```
# Spectrogram check at Fz (channel 0): AI vs Human
FZ_IDX = CH_NAMES.index('Fz')


def avg_spectrogram(epochs, fs):
    specs = []
    for e in epochs:
        f, tt, sxx = spectrogram(e[FZ_IDX], fs=fs, nperseg=min(256, e.
 ↪shape[-1]), noverlap=min(192, e.shape[-1] - 1))
        specs.append(sxx)
```

```
    return f, tt, np.mean(specs, axis=0)

# Use fixed acquisition fs
plot_fs = TARGET_FS

X_h = X_epochs[y == 0]
X_ai = X_epochs[y == 1]

f, tt, S_h = avg_spectrogram(X_h, plot_fs)
_, _, S_ai = avg_spectrogram(X_ai, plot_fs)

fmask = (f >= 1) & (f <= 40)

db_h = 10 * np.log10(S_h[fmask] + 1e-12)
db_ai = 10 * np.log10(S_ai[fmask] + 1e-12)
db_diff = db_ai - db_h  # AI - Human

fig, axes = plt.subplots(1, 3, figsize=(18, 4), sharey=True)

im0 = axes[0].pcolormesh(tt, f[fmask], db_h, shading='gouraud', cmap='viridis')
axes[0].set_title('Human | Fz spectrogram (dB)')
axes[0].set_xlabel('Time (s)')
axes[0].set_ylabel('Freq (Hz)')

im1 = axes[1].pcolormesh(tt, f[fmask], db_ai, shading='gouraud', cmap='viridis')
axes[1].set_title('AI | Fz spectrogram (dB)')
axes[1].set_xlabel('Time (s)')

v = np.max(np.abs(db_diff))
im2 = axes[2].pcolormesh(tt, f[fmask], db_diff, shading='gouraud',␣
 ↪cmap='RdBu_r', vmin=-v, vmax=v)
axes[2].set_title('AI - Human | Fz spectrogram (dB)')
axes[2].set_xlabel('Time (s)')

fig.colorbar(im1, ax=axes[:2], shrink=0.8, label='dB')
fig.colorbar(im2, ax=axes[2], shrink=0.8, label='dB diff')
plt.tight_layout()
plt.show()

# Optional quick scalar check in theta band over entire epoch
theta_mask = (f >= 4) & (f <= 8)
print('Mean Fz theta spectrogram power (dB):')
print(' Human:', db_h[theta_mask[fmask]].mean())
print(' AI   :', db_ai[theta_mask[fmask]].mean())
```

```
C:\Users\abepa\AppData\Local\Temp\ipykernel_74104\1690262027.py:45: UserWarning:
This figure includes Axes that are not compatible with tight_layout, so results
might be incorrect.
```

```
plt.tight_layout()
```




```
Mean Fz theta spectrogram power (dB):
 Human: 4.59278277766132
 AI   : 3.5568987917637074
```

[8]:
```
# Classification: AI vs Human with independent final holdout
feature_cols = [c for c in feat_df.columns if any(c.endswith('_' + b) for b in␣
 ↪BANDS.keys())]
X = feat_df[feature_cols].to_numpy()
y = feat_df['y'].to_numpy()

if 'train_idx' not in globals() or 'final_idx' not in globals():
    idx_all = np.arange(len(y))
    train_idx, final_idx = train_test_split(
        idx_all, test_size=FINAL_TEST_SIZE, stratify=y,␣
 ↪random_state=RANDOM_STATE, shuffle=True
    )

X_dev, y_dev = X[train_idx], y[train_idx]
X_final, y_final = X[final_idx], y[final_idx]

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LinearDiscriminantAnalysis()),
])

scoring = {'acc': 'accuracy', 'auc': 'roc_auc', 'f1': 'f1'}

# Development-only CV
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
res_dev = cross_validate(pipe, X_dev, y_dev, cv=skf, scoring=scoring)
print('Development CV (Stratified 5-fold):')
for k in scoring:
    print(f"  {k}: {res_dev['test_' + k].mean():.3f} ± {res_dev['test_' + k].
 ↪std():.3f}")
```

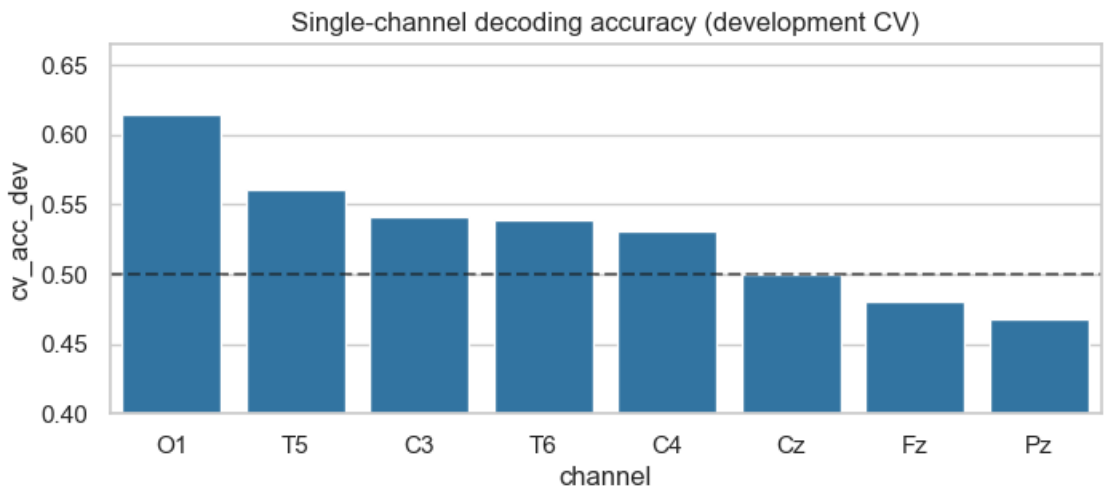
```
# Final independent test (single-shot evaluation)
pipe.fit(X_dev, y_dev)
y_pred_final = pipe.predict(X_final)
y_score_final = pipe.predict_proba(X_final)[:, 1]
print('\nFinal independent test:')
print(f"  acc: {accuracy_score(y_final, y_pred_final):.3f}")
print(f"  auc: {roc_auc_score(y_final, y_score_final):.3f}")
print(f"  f1 : {f1_score(y_final, y_pred_final):.3f}")
```

```
Development CV (Stratified 5-fold):
  acc: 0.417 ± 0.075
  auc: 0.399 ± 0.097
  f1: 0.445 ± 0.128

Final independent test:
  acc: 0.583
  auc: 0.597
  f1 : 0.583
```

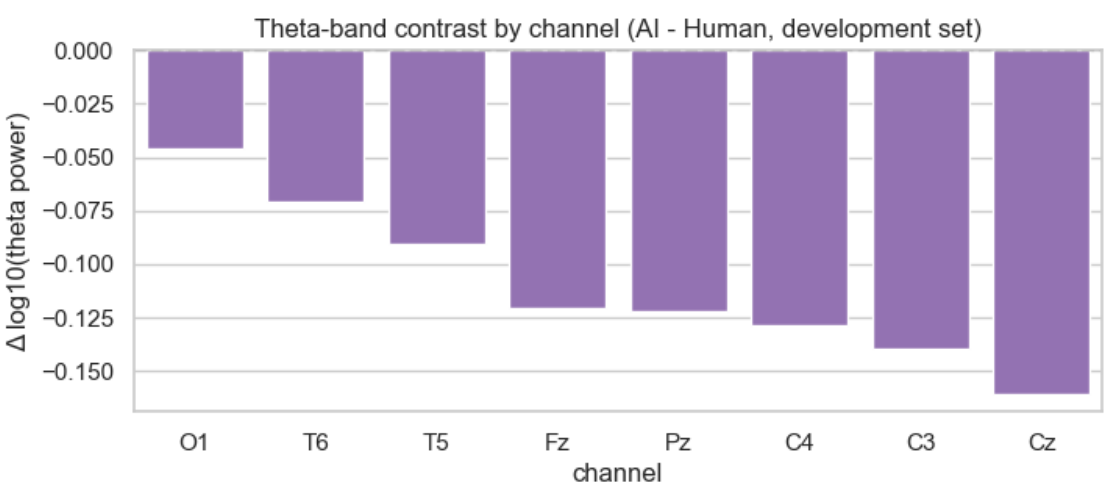
[9]:
```
# Region/channel contribution (development CV only to avoid final-test leakage)
if 'train_idx' not in globals() or 'final_idx' not in globals():
    idx_all = np.arange(len(y))
    train_idx, final_idx = train_test_split(
        idx_all, test_size=FINAL_TEST_SIZE, stratify=y,␣
 ↪random_state=RANDOM_STATE, shuffle=True
    )

y_dev = feat_df.iloc[train_idx]['y'].to_numpy()
skf_ch = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

# A) Single-channel decoding using all bands from that channel
ch_acc = []
for ch in CH_NAMES:
    ch_cols = [f'{ch}_{b}' for b in BANDS.keys()]
    X_ch_dev = feat_df.iloc[train_idx][ch_cols].to_numpy()
    sc = cross_val_score(Pipeline([
        ('scaler', StandardScaler()),
        ('clf', LinearDiscriminantAnalysis())
    ]), X_ch_dev, y_dev, cv=skf_ch, scoring='accuracy').mean()
    ch_acc.append({'channel': ch, 'cv_acc_dev': sc})

ch_acc_df = pd.DataFrame(ch_acc).sort_values('cv_acc_dev', ascending=False)
display(ch_acc_df)

plt.figure(figsize=(8, 3))
sns.barplot(data=ch_acc_df, x='channel', y='cv_acc_dev', color='tab:blue')
plt.axhline(0.5, ls='--', c='k', alpha=0.6)
```

```
plt.ylim(0.4, max(0.55, ch_acc_df['cv_acc_dev'].max() + 0.05))
plt.title('Single-channel decoding accuracy (development CV)')
plt.show()

# B) Channel-wise theta contrast (AI - Human) on development only
feat_dev = feat_df.iloc[train_idx].copy()
theta_delta = []
for ch in CH_NAMES:
    ai_m = np.log10(feat_dev.loc[feat_dev['category'] == 'AI', f'{ch}_theta'] +␣
 ↪1e-12).mean()
    hu_m = np.log10(feat_dev.loc[feat_dev['category'] == 'Human',␣
 ↪f'{ch}_theta'] + 1e-12).mean()
    theta_delta.append({'channel': ch, 'theta_log_power_ai_minus_human_dev':␣
 ↪ai_m - hu_m})

theta_delta_df = pd.DataFrame(theta_delta).
 ↪sort_values('theta_log_power_ai_minus_human_dev', ascending=False)
display(theta_delta_df)

plt.figure(figsize=(8, 3))
sns.barplot(data=theta_delta_df, x='channel',␣
 ↪y='theta_log_power_ai_minus_human_dev', color='tab:purple')
plt.axhline(0, ls='--', c='k', alpha=0.6)
plt.title('Theta-band contrast by channel (AI - Human, development set)')
plt.ylabel('Δ log10(theta power)')
plt.show()

print('Top channel(s) by development decoding:', ', '.join(ch_acc_df.
 ↪head(3)['channel'].tolist()))
print('Top channel(s) by development theta contrast:', ', '.join(theta_delta_df.
 ↪head(3)['channel'].tolist()))
```

```
  channel  cv_acc_dev
7      O1    0.614737
4      T5    0.561053
1      C3    0.541579
6      T6    0.538947
3      C4    0.531053
2      Cz    0.500000
0      Fz    0.480000
5      Pz    0.468421
```



	channel	theta_log_power_ai_minus_human_dev
7	O1	-0.045607
6	T6	-0.071029
4	T5	-0.090962
0	Fz	-0.120750
5	Pz	-0.122197
3	C4	-0.128973
1	C3	-0.139590
2	Cz	-0.161150




```
Top channel(s) by development decoding: O1, T5, C3
Top channel(s) by development theta contrast: O1, T6, T5
```

[ ]:
```
# ERP/time-locked signature analysis (retry with clearer event-locking)

if len(X_epochs) == 0:
    raise RuntimeError('No epochs available. Run dataset build cell first.')

# Conditions
X_h = X_epochs[y == 0]  # Human
X_ai = X_epochs[y == 1] # AI

t = np.arange(X_epochs.shape[-1]) / TARGET_FS

# Event anchors (seconds)
stim_onset = 0.0
motion_onset = 1.0  # user-specified anchor for wince/artificial motion

# Channel groups
idx_t56 = [CH_NAMES.index('T5'), CH_NAMES.index('T6')]
idx_czfz = [CH_NAMES.index('Cz'), CH_NAMES.index('Fz')]
idx_pzcz = [CH_NAMES.index('Pz'), CH_NAMES.index('Cz')]
idx_pz = [CH_NAMES.index('Pz')]
idx_c34 = [CH_NAMES.index('C3'), CH_NAMES.index('C4')]


def win_mask(tvec, t0, t1):
    return (tvec >= t0) & (tvec <= t1)


def baseline_correct(epochs, tvec, baseline_window, ch_idx):
    """Baseline-correct selected channels per trial and return mean signal␣
 ↪(n_trials, n_time)."""
    bmask = win_mask(tvec, baseline_window[0], baseline_window[1])
    sig = epochs[:, ch_idx, :].mean(axis=1)
    base = sig[:, bmask].mean(axis=1, keepdims=True)
    return sig - base


def peak_metrics(sig_trials, tvec, t0, t1, polarity='neg'):
    """Peak amplitude + latency in window from baseline-corrected trial␣
 ↪waveforms."""
    m = win_mask(tvec, t0, t1)
    data = sig_trials[:, m]
    tw = tvec[m]

    if polarity == 'neg':
        pk_idx = np.argmin(data, axis=1)
    else:
        pk_idx = np.argmax(data, axis=1)
```

```
    amps = data[np.arange(data.shape[0]), pk_idx]
    lats = tw[pk_idx]
    return amps, lats


# ============================
# 1) N170 (stimulus-locked)
# ============================
# Baseline near trial start (we do not have true pre-stim data)
n170_baseline = (0.00, 0.10)
n170_win = (0.12, 0.24)

sig_h_n170 = baseline_correct(X_h, t, n170_baseline, idx_t56)
sig_ai_n170 = baseline_correct(X_ai, t, n170_baseline, idx_t56)

n170_h_amp, n170_h_lat = peak_metrics(sig_h_n170, t, *n170_win, polarity='neg')
n170_ai_amp, n170_ai_lat = peak_metrics(sig_ai_n170, t, *n170_win,␣
 ↪polarity='neg')
n170_lag_ms = (np.mean(n170_ai_lat) - np.mean(n170_h_lat)) * 1000.0  # AI-Human


# ============================================
# 2) Motion-locked N2 / N400 / LPP (wince=1.0s)
# ============================================
# Use pre-motion baseline so post-motion components are easier to interpret.
motion_baseline = (0.80, 1.00)

sig_h_n2 = baseline_correct(X_h, t, motion_baseline, idx_czfz)
sig_ai_n2 = baseline_correct(X_ai, t, motion_baseline, idx_czfz)
n2_win = (1.20, 1.30)
n2_h_amp, n2_h_lat = peak_metrics(sig_h_n2, t, *n2_win, polarity='neg')
n2_ai_amp, n2_ai_lat = peak_metrics(sig_ai_n2, t, *n2_win, polarity='neg')

sig_h_n400 = baseline_correct(X_h, t, motion_baseline, idx_pzcz)
sig_ai_n400 = baseline_correct(X_ai, t, motion_baseline, idx_pzcz)
n400_win = (1.35, 1.50)
n400_h_amp, n400_h_lat = peak_metrics(sig_h_n400, t, *n400_win, polarity='neg')
n400_ai_amp, n400_ai_lat = peak_metrics(sig_ai_n400, t, *n400_win,␣
 ↪polarity='neg')

sig_h_lpp = baseline_correct(X_h, t, motion_baseline, idx_pz)
sig_ai_lpp = baseline_correct(X_ai, t, motion_baseline, idx_pz)
lpp_win = (1.60, 5.00)
lpp_mask = win_mask(t, *lpp_win)
lpp_h = sig_h_lpp[:, lpp_mask].mean(axis=1)
lpp_ai = sig_ai_lpp[:, lpp_mask].mean(axis=1)
```

```python
# ============================
# 3) Mu suppression (C3/C4)
# ============================
def moving_mu_power_db(epochs, ch_idx, fs, band=(8, 13), win_sec=0.40,␣
↪step_sec=0.04, baseline=(0.80, 1.00)):
    sig = epochs[:, ch_idx, :].mean(axis=1)
    sos = butter(4, [band[0] / (fs / 2.0), band[1] / (fs / 2.0)], btype='band',␣
↪output='sos')
    sig_f = sosfiltfilt(sos, sig, axis=1)

    w = int(round(win_sec * fs))
    step = max(1, int(round(step_sec * fs)))
    starts = np.arange(0, sig_f.shape[1] - w + 1, step)
    centers_t = (starts + w // 2) / fs

    pwr = []
    for s in starts:
        seg = sig_f[:, s:s + w]
        pwr.append(np.mean(seg ** 2, axis=1))
    pwr = np.stack(pwr, axis=1)

    bmask = (centers_t >= baseline[0]) & (centers_t <= baseline[1])
    base = pwr[:, bmask].mean(axis=1, keepdims=True)

    # Negative dB indicates suppression relative to pre-motion baseline
    pwr_db = 10 * np.log10((pwr + 1e-12) / (base + 1e-12))
    return centers_t, pwr_db


mu_t_h, mu_db_h = moving_mu_power_db(X_h, idx_c34, TARGET_FS)
mu_t_ai, mu_db_ai = moving_mu_power_db(X_ai, idx_c34, TARGET_FS)

mu_task_mask = (mu_t_h >= 1.0) & (mu_t_h <= 5.0)
mu_h_task = mu_db_h[:, mu_task_mask].mean(axis=1)
mu_ai_task = mu_db_ai[:, mu_task_mask].mean(axis=1)

# Max suppression around 2.5 s neighborhood
mu_peak_win = (2.2, 2.8)
peak_mask = (mu_t_h >= mu_peak_win[0]) & (mu_t_h <= mu_peak_win[1])
mu_h_peak = mu_db_h[:, peak_mask].min(axis=1)
mu_ai_peak = mu_db_ai[:, peak_mask].min(axis=1)


# ============================
# Summary table
```

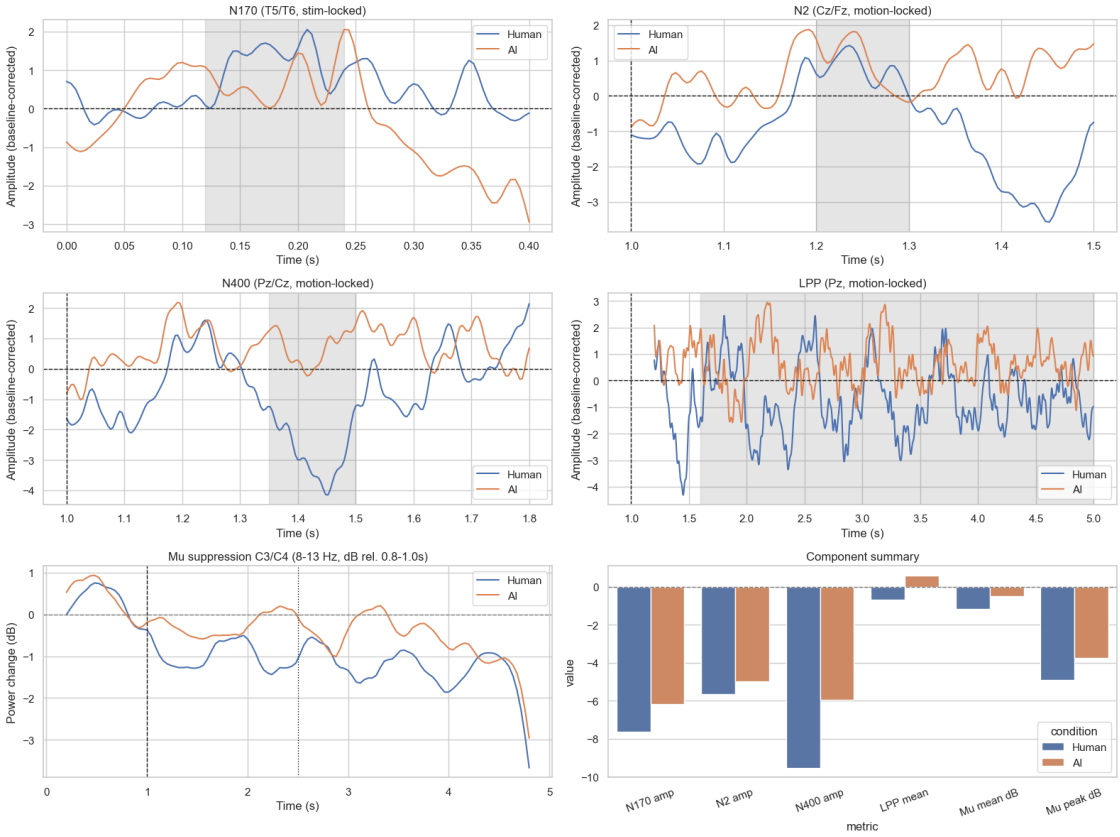
```python
# ============================
rows = [
    {
        'component': 'N170 (T5/T6)',
        'window_s': f'{n170_win[0]:.2f}-{n170_win[1]:.2f}',
        'human_amp_mean': np.mean(n170_h_amp),
        'ai_amp_mean': np.mean(n170_ai_amp),
        'human_lat_ms': np.mean(n170_h_lat) * 1000,
        'ai_lat_ms': np.mean(n170_ai_lat) * 1000,
        'p_amp': ttest_ind(n170_ai_amp, n170_h_amp, equal_var=False).pvalue,
        'p_latency': ttest_ind(n170_ai_lat, n170_h_lat, equal_var=False).pvalue,
    },
    {
        'component': 'N2 (Cz/Fz)',
        'window_s': f'{n2_win[0]:.2f}-{n2_win[1]:.2f}',
        'human_amp_mean': np.mean(n2_h_amp),
        'ai_amp_mean': np.mean(n2_ai_amp),
        'human_lat_ms': np.mean(n2_h_lat) * 1000,
        'ai_lat_ms': np.mean(n2_ai_lat) * 1000,
        'p_amp': ttest_ind(n2_ai_amp, n2_h_amp, equal_var=False).pvalue,
        'p_latency': ttest_ind(n2_ai_lat, n2_h_lat, equal_var=False).pvalue,
    },
    {
        'component': 'N400 (Pz/Cz)',
        'window_s': f'{n400_win[0]:.2f}-{n400_win[1]:.2f}',
        'human_amp_mean': np.mean(n400_h_amp),
        'ai_amp_mean': np.mean(n400_ai_amp),
        'human_lat_ms': np.mean(n400_h_lat) * 1000,
        'ai_lat_ms': np.mean(n400_ai_lat) * 1000,
        'p_amp': ttest_ind(n400_ai_amp, n400_h_amp, equal_var=False).pvalue,
        'p_latency': ttest_ind(n400_ai_lat, n400_h_lat, equal_var=False).pvalue,
    },
    {
        'component': 'LPP (Pz)',
        'window_s': f'{lpp_win[0]:.2f}-{lpp_win[1]:.2f}',
        'human_amp_mean': np.mean(lpp_h),
        'ai_amp_mean': np.mean(lpp_ai),
        'human_lat_ms': np.nan,
        'ai_lat_ms': np.nan,
        'p_amp': ttest_ind(lpp_ai, lpp_h, equal_var=False).pvalue,
        'p_latency': np.nan,
    },
    {
        'component': 'Mu suppression mean (C3/C4, dB)',
        'window_s': '1.00-5.00',
        'human_amp_mean': np.mean(mu_h_task),
        'ai_amp_mean': np.mean(mu_ai_task),
```

```
        'human_lat_ms': np.nan,
        'ai_lat_ms': np.nan,
        'p_amp': ttest_ind(mu_ai_task, mu_h_task, equal_var=False).pvalue,
        'p_latency': np.nan,
    },
    {
        'component': 'Mu suppression peak (C3/C4, dB)',
        'window_s': f'{mu_peak_win[0]:.2f}-{mu_peak_win[1]:.2f}',
        'human_amp_mean': np.mean(mu_h_peak),
        'ai_amp_mean': np.mean(mu_ai_peak),
        'human_lat_ms': np.nan,
        'ai_lat_ms': np.nan,
        'p_amp': ttest_ind(mu_ai_peak, mu_h_peak, equal_var=False).pvalue,
        'p_latency': np.nan,
    },
]

sig_df = pd.DataFrame(rows)
display(sig_df)

print(f'N170 processing lag (AI - Human) = {n170_lag_ms:.1f} ms')
print('Interpretation note: negative amplitudes are expected for N170/N2/N400;')
print('mu suppression is expected as negative dB relative to pre-motion␣
 ↪baseline.')


# ============================
# Plots
# ============================
fig, axes = plt.subplots(3, 2, figsize=(16, 12), sharex=False)

# N170
m = win_mask(t, 0.0, 0.4)
axes[0, 0].plot(t[m], sig_h_n170.mean(axis=0)[m], label='Human')
axes[0, 0].plot(t[m], sig_ai_n170.mean(axis=0)[m], label='AI')
axes[0, 0].axvspan(*n170_win, color='gray', alpha=0.2)
axes[0, 0].axhline(0, c='k', ls='--', lw=1)
axes[0, 0].set_title('N170 (T5/T6, stim-locked)')
axes[0, 0].set_xlabel('Time (s)')
axes[0, 0].set_ylabel('Amplitude (baseline-corrected)')
axes[0, 0].legend()

# N2
m = win_mask(t, 1.0, 1.5)
axes[0, 1].plot(t[m], sig_h_n2.mean(axis=0)[m], label='Human')
axes[0, 1].plot(t[m], sig_ai_n2.mean(axis=0)[m], label='AI')
axes[0, 1].axvspan(*n2_win, color='gray', alpha=0.2)
```

```
axes[0, 1].axvline(motion_onset, c='k', ls='--', lw=1)
axes[0, 1].axhline(0, c='k', ls='--', lw=1)
axes[0, 1].set_title('N2 (Cz/Fz, motion-locked)')
axes[0, 1].set_xlabel('Time (s)')
axes[0, 1].set_ylabel('Amplitude (baseline-corrected)')
axes[0, 1].legend()

# N400
m = win_mask(t, 1.0, 1.8)
axes[1, 0].plot(t[m], sig_h_n400.mean(axis=0)[m], label='Human')
axes[1, 0].plot(t[m], sig_ai_n400.mean(axis=0)[m], label='AI')
axes[1, 0].axvspan(*n400_win, color='gray', alpha=0.2)
axes[1, 0].axvline(motion_onset, c='k', ls='--', lw=1)
axes[1, 0].axhline(0, c='k', ls='--', lw=1)
axes[1, 0].set_title('N400 (Pz/Cz, motion-locked)')
axes[1, 0].set_xlabel('Time (s)')
axes[1, 0].set_ylabel('Amplitude (baseline-corrected)')
axes[1, 0].legend()

# LPP
m = win_mask(t, 1.2, 5.0)
axes[1, 1].plot(t[m], sig_h_lpp.mean(axis=0)[m], label='Human')
axes[1, 1].plot(t[m], sig_ai_lpp.mean(axis=0)[m], label='AI')
axes[1, 1].axvspan(*lpp_win, color='gray', alpha=0.2)
axes[1, 1].axvline(motion_onset, c='k', ls='--', lw=1)
axes[1, 1].axhline(0, c='k', ls='--', lw=1)
axes[1, 1].set_title('LPP (Pz, motion-locked)')
axes[1, 1].set_xlabel('Time (s)')
axes[1, 1].set_ylabel('Amplitude (baseline-corrected)')
axes[1, 1].legend()

# Mu suppression trajectory
axes[2, 0].plot(mu_t_h, mu_db_h.mean(axis=0), label='Human')
axes[2, 0].plot(mu_t_ai, mu_db_ai.mean(axis=0), label='AI')
axes[2, 0].axvline(1.0, c='k', ls='--', lw=1)
axes[2, 0].axvline(2.5, c='k', ls=':', lw=1)
axes[2, 0].axhline(0, c='gray', ls='--', lw=1)
axes[2, 0].set_title('Mu suppression C3/C4 (8-13 Hz, dB rel. 0.8-1.0s)')
axes[2, 0].set_xlabel('Time (s)')
axes[2, 0].set_ylabel('Power change (dB)')
axes[2, 0].legend()

# Summary bars
summary_plot = pd.DataFrame({
    'metric': ['N170 amp', 'N2 amp', 'N400 amp', 'LPP mean', 'Mu mean dB', 'Mu␣
 ↪peak dB'],
```

```
    'Human': [np.mean(n170_h_amp), np.mean(n2_h_amp), np.mean(n400_h_amp), np.
 ↪mean(lpp_h), np.mean(mu_h_task), np.mean(mu_h_peak)],
    'AI':    [np.mean(n170_ai_amp), np.mean(n2_ai_amp), np.mean(n400_ai_amp),␣
 ↪np.mean(lpp_ai), np.mean(mu_ai_task), np.mean(mu_ai_peak)],
})
summary_plot = summary_plot.melt(id_vars='metric', var_name='condition',␣
 ↪value_name='value')
sns.barplot(data=summary_plot, x='metric', y='value', hue='condition',␣
 ↪ax=axes[2, 1])
axes[2, 1].axhline(0, c='gray', ls='--', lw=1)
axes[2, 1].set_title('Component summary')
axes[2, 1].tick_params(axis='x', rotation=20)

plt.tight_layout()
plt.show()
```

```
                        component   window_s  human_amp_mean  ai_amp_mean  \
0                    N170 (T5/T6)  0.12-0.24       -7.641443    -6.177599
1                      N2 (Cz/Fz)  1.20-1.30       -5.633645    -4.966564
2                    N400 (Pz/Cz)  1.35-1.50       -9.530390    -5.947977
3                        LPP (Pz)  1.60-5.00       -0.689647     0.597528
4  Mu suppression mean (C3/C4, dB)  1.00-5.00       -1.184963    -0.491221
5  Mu suppression peak (C3/C4, dB)  2.20-2.80       -4.899827    -3.740703

   human_lat_ms  ai_lat_ms     p_amp  p_latency
0    180.733333      185.8  0.309163   0.523718
1   1251.200000     1257.4  0.734071   0.375204
2   1427.200000     1428.8  0.011530   0.854920
3           NaN        NaN  0.170822        NaN
4           NaN        NaN  0.227065        NaN
5           NaN        NaN  0.138183        NaN

N170 processing lag (AI - Human) = 5.1 ms
Interpretation note: negative amplitudes are expected for N170/N2/N400;
mu suppression is expected as negative dB relative to pre-motion baseline.
```



1.1 Targeted ERP / time-locked signatures

Below we test the specific signatures you requested using fixed windows on the 5 s epochs.

Assumptions used in this notebook: - Stimulus/video onset is at $t = 0$ s (epoch start). - Wince/artificial-motion anchor is approximately at $t = 1.0$ s.

Component checks: - **Visual N170** at **T5/T6** around $t = 0.17$ s (negative peak, and AI-vs-Human latency lag). - **Empathy N2** at **Cz/Fz** around $t = 1.2$–1.3 s (negative peak after wince onset). - **Uncanny N400** at **Pz/Cz** around $t \approx 1.4$ s (negative error-like deflection). - **Sustained LPP** at **Pz** from $t = 1.6$ to 5.0 s (positive sustained potential). - **Mu suppression** at **C3/C4** (8–13 Hz), expected drop from about 1.0 s onward, strongest near 2.5 s.

1.2 Interpretation guide

- If **Fz theta (AI) > Fz theta (Human)** and the effect is statistically reliable, that supports your hypothesis of stronger frontal theta recruitment during AI/deepfake-like processing.
- If channel-ranking consistently highlights **Fz/Cz/C3/C4**, it suggests frontal/sensorimotor contribution.
- If posterior channels (e.g., O1) dominate, differences may be more visual-perceptual than frontal-evaluative.

Caveats: - Current analysis is trial-level and observational; causal claims need more controls. -

Subject count is still small, so subject-wise CV is the more trustworthy metric. - We repaired bad channels conservatively (median of good channels). Consider ICA or RANSAC for a stronger artifact pipeline in a follow-up.